

CUSTO DAS ABSTRAÇÕES IDIOMÁTICAS NA JVM: UM ESTUDO QUANTITATIVO COMPARANDO IMPLEMENTAÇÕES JAVA E CLOJURE NO PROCESSAMENTO DE REQUISIÇÕES HTTP

NATHAN PAIVA DE LACERDA¹, JULIANO ZANUZZIO BLANCO²

RESUMO: O presente artigo, apresentado como Trabalho de Conclusão do Curso de Engenharia da Computação do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), *Câmpus Piracicaba*, trata do desenvolvimento de um estudo quantitativo sobre o custo das abstrações idiomáticas do Paradigma Funcional na *Java Virtual Machine* (JVM), comparando três implementações de um microsserviço de cálculo de juros compostos: *Java Orientado a Objetos*, *Clojure Idiomático* e *Clojure com interoperabilidade Java*. O objetivo foi mensurar o impacto dessas abstrações sobre a latência e a estabilidade do processamento de requisições HTTP sob diferentes níveis de carga, investigando o papel mediador do comportamento do *Garbage Collector*. Para tanto, foram conduzidos testes de carga em três níveis — 100, 500 e 1000 requisições por segundo —, com coleta de métricas de latência, *throughput* e comportamento do coletor de lixo *G1GC*, complementados por análise comparativa do *bytecode* gerado por cada implementação e validação estatística por meio do Coeficiente de Variação (CV%). Os resultados mostraram que, sob baixa e média carga, o *Java* apresentou a menor latência de cauda, enquanto o *Clojure Idiomático* manteve menor pressão sobre o coletor de lixo; já sob saturação, a 1000 requisições por segundo, as latências médias convergiram entre as três implementações, mas o *Java* exibiu instabilidade estrutural significativamente maior entre execuções (CV% de 31,14% para o percentil 95, contra 13,05% do *Clojure Idiomático* e 15,73% do *Clojure Interop*). Esse comportamento foi atribuído à promoção prematura de objetos mutáveis para a *Old Generation* sob alta taxa de injeção, em contraste com o alinhamento dos objetos imutáveis do *Clojure Idiomático* à *hipótese geracional fraca* do coletor *G1GC*. Conclui-se que o custo das abstrações funcionais

¹Graduando em Engenharia da Computação, IFSP, Câmpus Piracicaba, nathan.paiva@aluno.ifsp.edu.br

² Docente, IFSP, Câmpus Piracicaba, julianoblanco@ifsp.edu.br

não reside na latência das execuções estáveis, mas na maior previsibilidade e estabilidade que a imutabilidade proporciona sob saturação.

Palavras-chave: Máquina Virtual Java; Clojure; Paradigmas de Programação; Coletor de Lixo; Desempenho de Software.

***ABSTRACT:** This article, presented as the Final Project for the Computer Engineering degree at the Federal Institute of Education, Science and Technology of São Paulo (IFSP), Piracicaba Campus, discusses the development of a quantitative study on the cost of idiomatic abstractions from the Functional Paradigm on the Java Virtual Machine (JVM), comparing three implementations of a compound interest microservice: Object-Oriented Java, Idiomatic Clojure, and Clojure with Java interoperability. The goal was to measure the impact of these abstractions on the latency and stability of HTTP request processing under different load levels, investigating the mediating role of Garbage Collector behavior. To this end, load tests were conducted at three levels — 100, 500, and 1000 requests per second —, collecting latency, throughput, and G1GC behavior metrics, complemented by a comparative bytecode analysis of each implementation and statistical validation through the Coefficient of Variation (CV%). Results showed that, under low and moderate load, Java presented the lowest tail latency, while Idiomatic Clojure maintained lower pressure on the garbage collector; under saturation, at 1000 requests per second, average latencies converged across the three implementations, but Java exhibited significantly greater structural instability between runs (CV% of 31.14% for the 95th percentile, versus 13.05% for Idiomatic Clojure and 15.73% for Clojure Interop). This behavior was attributed to the premature promotion of mutable objects to the Old Generation under high injection rates, in contrast to the alignment of Idiomatic Clojure's immutable objects with the G1GC collector's weak generational hypothesis. It is concluded that the cost of functional abstractions does not lie in the latency of stable executions, but in the greater predictability and stability that immutability provides under saturation.*

Keywords: Java Virtual Machine; Clojure; Programming Paradigms; Garbage Collector; Software Performance.

1. INTRODUÇÃO

1.1. CONTEXTUALIZAÇÃO

A *Java Virtual Machine* (JVM) é uma máquina virtual que abstrai o *hardware* subjacente, executando *bytecode* de forma independente da linguagem que o gerou. Embora originalmente projetada para a linguagem Java, a JVM evoluiu arquiteturalmente para suportar múltiplas linguagens, tornando-se uma plataforma de execução compartilhada por paradigmas distintos (ROSE, 2009). Essa capacidade multilinguagem é empiricamente documentada na literatura por meio da caracterização dinâmica de seis linguagens distintas e hospedadas sobre esse ecossistema – Java, Scala, Clojure, Jython, JRuby e JavaScript –, estudo que evidencia que linguagens pertencentes a paradigmas diferentes produzem perfis de execução mensuravelmente distintos sobre a mesma plataforma (SARIMBEKOV et al., 2016). Essa distinção tem implicações diretas de desempenho, as quais constituem o objeto de investigação do presente trabalho.

1.2. PROBLEMA DA PESQUISA E HIPÓTESE

Tendo Java como uma linguagem de programação representante do *Paradigma Orientado a Objetos* (POO) e Clojure como representante do *Paradigma Funcional* (PF), detalhados e explicados com mais profundidade na próxima seção de *Fundamentação Teórica*, o objeto de estudo deste trabalho tem como ponto de partida o ecossistema abrangente e solidificado da *Java Virtual Machine*, inicialmente desenvolvida com heurísticas de otimização voltadas para a sua linguagem de origem, o Java. Infere-se, portanto, naturalmente que linguagens capazes de rodar nesse ambiente e que possuem padrões de execução distintos obteriam perfis de *runtime* potencialmente prejudicados por tais heurísticas existentes (SARIMBEKOV et al., 2016).

Clojure é uma linguagem funcional centrada na imutabilidade e possui abstrações idiomáticas distintas de qualquer linguagem predominantemente regida pela POO. A partir dessas características, ao atuar em tal ambiente de desenvolvimento, Clojure gera abstrações funcionais que introduzem *overhead* mensurável na JVM, devido a alocações de objetos temporários extras e chamadas virtuais adicionais que não ocorrem em implementações imperativas equivalentes (MØLLER; VEILEBORG, 2020), logo produzindo uma taxa de alocação de objetos superior no *heap* e, como característica relevante, cada um destes objetos com um tempo de vida exíguo.

No entanto, partindo desse comportamento e da expectativa de *overhead* funcional, coloca-se como hipótese central deste trabalho que o padrão de alocação distinto produzido pelo Clojure Idiomático – objetos imutáveis de curta duração gerados iterativamente a cada requisição – pode se alinhar ao pressuposto fundamental sobre o qual coletores de lixo geracionais modernos foram projetados (LIEBERMAN; HEWITT, 1983; UNGAR, 1984), produzindo comportamento diferenciado do *Garbage Collector* (GC) e, conseqüentemente, perfis distintos de latência e estabilidade sob carga crescente.

Diante disso, este trabalho formula o seguinte questionamento de pesquisa: qual é o custo das abstrações idiomáticas geradas pelo Paradigma Funcional na JVM em um cenário *cliente-servidor* no que diz respeito ao processamento de requisições HTTP sob diferentes níveis de carga, e de que forma o comportamento do coletor de lixo medeia tanto a latência observada quanto a estabilidade das implementações sob saturação?

1.3. OBJETIVOS

De forma geral, o foco deste trabalho se embasa em mensurar e comparar de forma quantitativa o custo de desempenho das abstrações idiomáticas do Clojure frente ao Java no processamento de requisições HTTP sobre a JVM, além de investigar o papel do comportamento de uma das peças centrais para esse ecossistema de desenvolvimento, o *Garbage Collector*.

Para tanto, os seguintes objetivos específicos orientam o desenvolvimento do trabalho:

- Implementar três variantes de microsserviço para o processamento de requisições HTTP: Java Imperativo Orientado a Objetos, Clojure Idiomático e Clojure com Interoperabilidade Java;

- Executar testes de carga sob três níveis distintos, coletando métricas de latência, *throughput*, comportamento do GC e consumo de memória *heap*;
- Validar estatisticamente a reprodutibilidade dos resultados entre execuções por meio do *Coefficiente de Variação* (CV%);
- Analisar de forma geral o *bytecode* gerado por cada implementação, identificando diferenças estruturais nas instruções produzidas por cada paradigma;
- Correlacionar os padrões de *bytecode* identificados com os perfis de performance observados, investigando a cadeia causal entre paradigma, instruções geradas e comportamento de GC sob diferentes níveis de carga.

1.4. JUSTIFICATIVA

Inicialmente, vale ressaltar que a JVM é uma plataforma de produção amplamente adotada na indústria, na qual uma diversidade de linguagens cresceu significativamente sobre este sistema, abrangendo linguagens como Scala, Jython, JRuby, JavaScript (SARIMBEKOV et al., 2016), dentre outras. Somando-se a esse impacto de nível industrial e não só acadêmico, a grande maioria dos trabalhos existentes compara paradigmas em termos de produtividade, legibilidade ou desempenho bruto de algoritmos isolados, não em cenários de processamento de requisições com pressão de alocação real de coleta de lixo, abrangendo desde avaliações de métricas de qualidade de código entre implementações funcionais e orientadas a objetos (HARRISON et al., 1996; ALIC et al., 2016) até análises de características arquiteturais como extensibilidade, reusabilidade e tratamento de erros (SOUSA et al., 2025). Ademais, o resultado contraintuitivo de que o *overhead* mensurável de *bytecode* e de GC produzido pelo Clojure Idiomático não se traduz em penalidade de latência nas execuções estáveis, e de que o Java, paradoxalmente, demonstra maior instabilidade sob saturação, desafia a expectativa documentada na literatura, o que por si só justifica a investigação.

1.5. ORGANIZAÇÃO DO ARTIGO

O presente artigo está organizado em oito seções. A seção 2 apresenta a fundamentação teórica, abordando os paradigmas POO e PF, a JVM como plataforma multilinguagem e os conceitos de compilação e *bytecode*. A seção 3 discute os trabalhos relacionados, contextualizando o estudo frente à literatura existente. A seção 4 descreve a metodologia adotada, detalhando a arquitetura da solução, as implementações desenvolvidas, a infraestrutura de testes e o protocolo de execução. A seção 5 apresenta a análise do *bytecode* gerado por cada implementação, identificando diferenças estruturais entre os paradigmas. A seção 6 expõe os resultados dos testes de carga, cobrindo as análises de *throughput*, latência, comportamento do GC e validação estatística. A seção 7 discute os achados, correlacionando os padrões de *bytecode* com os perfis de performance observados e identificando limitações do estudo. Por fim, a seção 8 apresenta as considerações finais, sintetizando as contribuições do trabalho e apontando direções para trabalhos futuros.

2. FUNDAMENTAÇÃO TEÓRICA

Em termos abrangentes, um paradigma é um conjunto de princípios, restrições e abstrações que moldam o que é expressável e incentivado dentro de uma linguagem ou estilo de programação. De forma mais precisa, Robert W. Floyd transpõe explicitamente o conceito de paradigma cunhado por Thomas Kuhn em *The Structure of Scientific Revolutions (1962)* para a computação, definindo paradigma de programação não como um conjunto de recursos sintáticos de uma linguagem, mas como uma metodologia cognitiva de resolução de problemas – uma estrutura mental que organiza a forma como o programador raciocina sobre decomposição de problemas em partes menores, a relação entre essas partes, o controle do estado e o tratamento de efeitos colaterais (FLOYD, 1979).

2.1. PARADIGMA ORIENTADO A OBJETOS

Peter Wegner (1990) estabelece a definição formal e taxonômica do Paradigma Orientado a Objetos por meio de uma hierarquia tripartite: programas baseados em objetos são aqueles que encapsulam estado e comportamento; programas baseados em classes definem tipos de objetos por meio de classes; e programas orientados a objetos adicionam herança às classes. Ademais, a POO em sentido estrito requer os três elementos simultaneamente e, como essência do paradigma, tem-se o particionamento do estado de uma determinada aplicação em domínios locais encapsulados, nos quais a computação ocorre por meio da troca de mensagens entre objetos que modificam seu estado interno (WEGNER, 1990). Esse modelo centrado em mutação de estado encapsulado é o que distingue estruturalmente o Java do Clojure, em que o paradigma funcional rejeita a modificação de estado como mecanismo central de computação, conforme detalhado na seção que segue.

2.2. PARADIGMA FUNCIONAL

Enquanto o Paradigma Orientado a Objetos modela entidades do mundo real como objetos possuindo identidade, estado e comportamento, o Paradigma Funcional tem suas raízes no cálculo lambda de Alonzo Church, um sistema formal matemático no qual a computação é definida como avaliação de expressões e não como modificação sequencial de estado. Essa origem matemática é o que confere ao PF suas três propriedades centrais e indissociáveis: funções de primeira classe tratadas como dados, ausência de efeitos colaterais e imutabilidade de dados. A partir desses três traços estruturais, adquire-se garantia de que funções puras sempre produzirão o mesmo resultado para os mesmos argumentos, independentemente do histórico de execução, permitindo, dessa forma, o raciocínio equacional e verificação formal sobre programas (HUDAK, 1989). No contexto deste trabalho, é precisamente a imutabilidade que determina o padrão de alocação do Clojure na JVM: objetos imutáveis, uma vez criados, não podem ser modificados, o que resulta em objetos de vida curta que se alinham com as premissas sobre as quais coletores de lixo geracionais modernos foram projetados, argumento central desenvolvido nas seções subsequentes.

Linguagens imperativas tradicionais como o Java, no entanto, operam em um estilo *word-at-a-time* que espelha fisicamente a arquitetura de *von Neumann*: uma CPU que transfere uma palavra de dados por vez através de um barramento estreito entre processador e memória, configurando o que ficou conhecido como “*gargalo de von Neumann*”. John Warner Backus (1978) define tais linguagens como gordas e fracas de acordo com o trecho: “*Inherent defects at the most basic level cause them to be both fat and weak...*” [“Defeitos inerentes no nível mais básico fazem com que sejam ao mesmo tempo gordas e fracas...”, tradução nossa], sendo gordas porque crescem continuamente em tamanho sem ganho proporcional de poder expressivo, já que o acoplamento entre semântica e transição de estado obriga que cada detalhe de cada recurso seja embutido no próprio *framework* da linguagem; e fraca porque, apesar desse volume, carecem de propriedades matemáticas úteis para raciocinar sobre programas e são incapazes de usar formas combinatórias eficazes para construir programas novos a partir de existentes. O paradigma funcional surge, nesse contexto, justamente como caminho alternativo proposto por Backus: ao substituir atribuição e laços por composição de funções, o programa torna-se uma expressão matemática verificável sem dependência de histórico de execução (BACKUS, 1978).

2.3. A JVM COMO PLATAFORMA MULTILINGUAGEM

A Java Virtual Machine, bem como todo seu ecossistema estruturado, foi originalmente desenvolvida e implementada com foco na linguagem Java, de forma que suas heurísticas de otimização, tais como padrão de chamadas de métodos e pressão sobre o *Garbage Collector*, foram projetadas para os perfis de *runtime* característicos dessa linguagem. Nesse sentido, John R. Rose (2009) documenta, com propriedade de engenheiro de desenvolvimento da própria plataforma, que a JVM foi projetada para uma única linguagem e que, ao executar programas escritos em outras linguagens, surgem *pain points* [“pontos de dor”, tradução nossa] arquiteturais que prejudicam tanto o desenvolvimento quanto a execução. Ademais, tal argumento é corroborado empiricamente por Nakamura et al. (2024), os quais demonstraram que as heurísticas das JVMs modernas são implicitamente otimizadas para o *bytecode* gerado pelo compilador Java, penalizando estruturalmente linguagens que produzem padrões de instrução distintos.

Nesse contexto, Richard Hickey (2020), criador do Clojure, desenvolveu a hospedagem na JVM de forma intencional: o Clojure compila seu *código-fonte* diretamente para *bytecode* JVM, não como mera estratégia de implementação, mas como um compromisso de *design* que garante interoperabilidade direta com o ecossistema Java. Essa característica motivou, no presente estudo, o desenvolvimento de uma terceira variante do microserviço em Clojure com interoperabilidade Java, permitindo avaliar em que medida o uso de estruturas nativas Java dentro do código Clojure atenua os custos das abstrações idiomáticas funcionais.

2.4. COMPILAÇÃO E BYTECODE

Java é uma linguagem de tipagem estática, ou seja, os tipos de todas as variáveis e expressões são conhecidos e verificados em tempo de compilação, o que permite ao compilador gerar *bytecode* com instruções de invocação diretamente resolvidas e interpretadas pelo compilador JIT (*Just-In-Time*) da JVM. O Clojure, por sua vez, é uma linguagem de tipagem dinâmica, logo, na ausência de anotações de tipo explícitas, o compilador trata todos os valores como *Objects* por padrão, delegando a resolução de tipo para o *runtime*. Esse mecanismo implica o recurso à reflexão, ou seja, chamadas de método resolvidas via biblioteca *java.lang.reflect* em tempo de execução e o encapsulamento de tipos primitivos em objetos *Wrapper*, como *Integer* e *Long*, exigindo operações de *unboxing* em contextos aritméticos e gerando pressão adicional sobre o *heap* (HICKEY, 2020). Como consequência, linguagens dinamicamente tipadas como o Clojure geram *bytecode* com alta densidade de instruções dinâmicas, comprometendo o desempenho em tempo de execução em comparação a implementações estaticamente tipadas equivalentes (ALI et al., 2021).

Esses padrões distintos de geração de *bytecode* entre Java e Clojure constituem o objeto central da análise conduzida na seção 5 deste trabalho, na qual os *bytecodes* gerados por cada implementação são inspecionados e comparados com o objetivo de identificar diferenças estruturais nas instruções produzidas por cada paradigma e correlacioná-las aos perfis de performance observados nos testes de carga.

3. TRABALHOS RELACIONADOS

A comparação empírica entre paradigmas de programação possui tradição consolidada na literatura, com estudos que implementam algoritmos equivalentes nos dois paradigmas e extraem métricas quantitativas controladas de dimensionamento de código em números de linhas totais, taxa de erros e esforço de manutenção, sem revelar diferenças estatisticamente significativas na maioria dessas métricas de qualidade entre as abordagens funcionais e orientadas a objetos (HARRISON et al., 1996). Em complemento, trabalhos mais recentes dessa tradição estendem a comparação para métricas de tempo de execução e consumo de memória em algoritmos clássicos tais como *ordenação por merge sort*, *geração de números primos* e o *problema das N rainhas*, demonstrando que implementações imperativas em Java apresentam tempo de execução substancialmente inferior, enquanto implementações puramente funcionais em Haskell consomem, em média, metade da memória, comportamento esse atribuído ao uso de dados imutáveis e à abordagem *stateless* característica do paradigma funcional (ALIC et al., 2016). Ademais, ao introduzir o ecossistema da *Java Virtual Machine* como plataforma de comparação, um estudo empírico controlado com implementações equivalentes em Java e *Scala* para software paralelo em processadores multinúcleo constatou que a performance de execução entre as duas linguagens é comparável, contrariando a expectativa de que o estilo funcional implique degradação de desempenho na plataforma (PANKRATIUS et al., 2012).

No entanto, nenhum desses trabalhos investiga o comportamento dos paradigmas em cenário de servidor HTTP sob carga sustentada, nem correlaciona o padrão de alocação de objetos ao comportamento do *Garbage Collector* em *runtime*. A caracterização dinâmica de linguagens hospedadas na JVM, incluindo explicitamente Clojure, revela que objetos gerados por linguagens não-Java tendem a ser menores e de vida útil consideravelmente mais curta do que os gerados por Java nativo (LI et al., 2013), evidenciando que o padrão de alocação distinto do paradigma funcional na JVM é empiricamente documentado, porém suas implicações para o comportamento do GC sob carga HTTP sustentada permanecem sem investigação. A relevância dessa lacuna é reforçada pelo fato de que o alinhamento entre tempo de vida dos objetos e o ciclo de vida das requisições HTTP é um problema de pesquisa ativo: um coletor de lixo orientado especificamente ao ciclo de vida de requisições, avaliado em cargas reais com servidor web *Tomcat* e ambiente de desenvolvimento *Spring*, demonstrou ser capaz de reduzir em até 32% o tempo de execução em relação ao *G1GC*, coletor de lixo geracional moderno, padrão do *OpenJDK*, com sobrecarga de rastreamento dinâmico inferior a 2% (QIU; BLACKBURN, 2025).

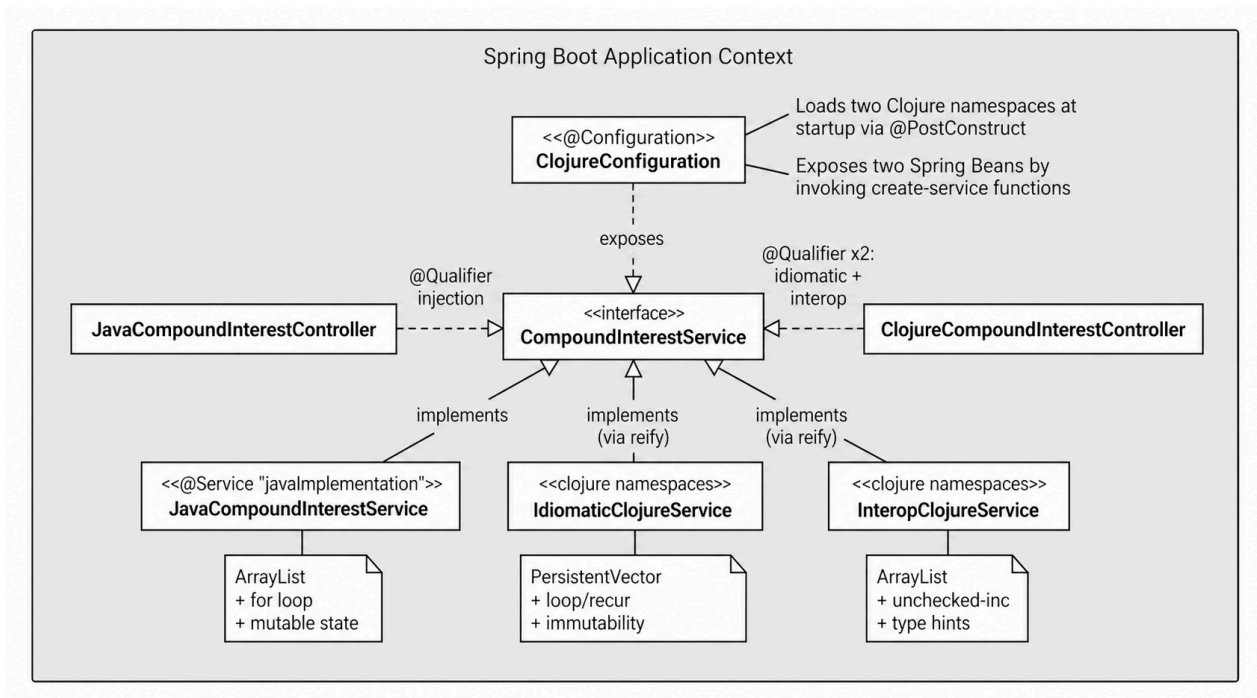
Por fim, o presente trabalho posiciona-se nessa lacuna, investigando, pelo ângulo da comparação de paradigmas de programação, se a imutabilidade idiomática do Clojure produz naturalmente o padrão de alocação que o coletor *G1GC* favorece, combinando análise de *bytecode* gerado por cada implementação e testes de carga sob três níveis de *throughput* em um cenário de microsserviço HTTP.

4. METODOLOGIA

4.1. ARQUITETURA DA SOLUÇÃO

A solução desenvolvida consiste em uma *API REST* construída sobre o *framework Spring Boot* e containerizada via *Docker Compose* em rede interna isolada. A API implementa uma arquitetura em camadas – *MVC (Model, View, Controller)*, na qual a *interface CompoundInterestService* define o contrato comum de negócio para o cálculo de juros compostos, implementado por três variantes de serviço: *JavaCompoundInterestService*, *compound_interest_service_idiomatic* e *compound_interest_service_interop_java*. A integração entre o ecossistema *Spring Boot* e as implementações Clojure é viabilizada pela classe *ClojureConfiguration*, responsável por carregar os *namespaces* Clojure na inicialização da aplicação e expô-los como *beans Spring*, permitindo que ambos os paradigmas operem sob o mesmo contrato de *interface*, conforme ilustrado na Figura 1.

Figura 1 – Diagrama de componentes: integração Spring Boot e Clojure.

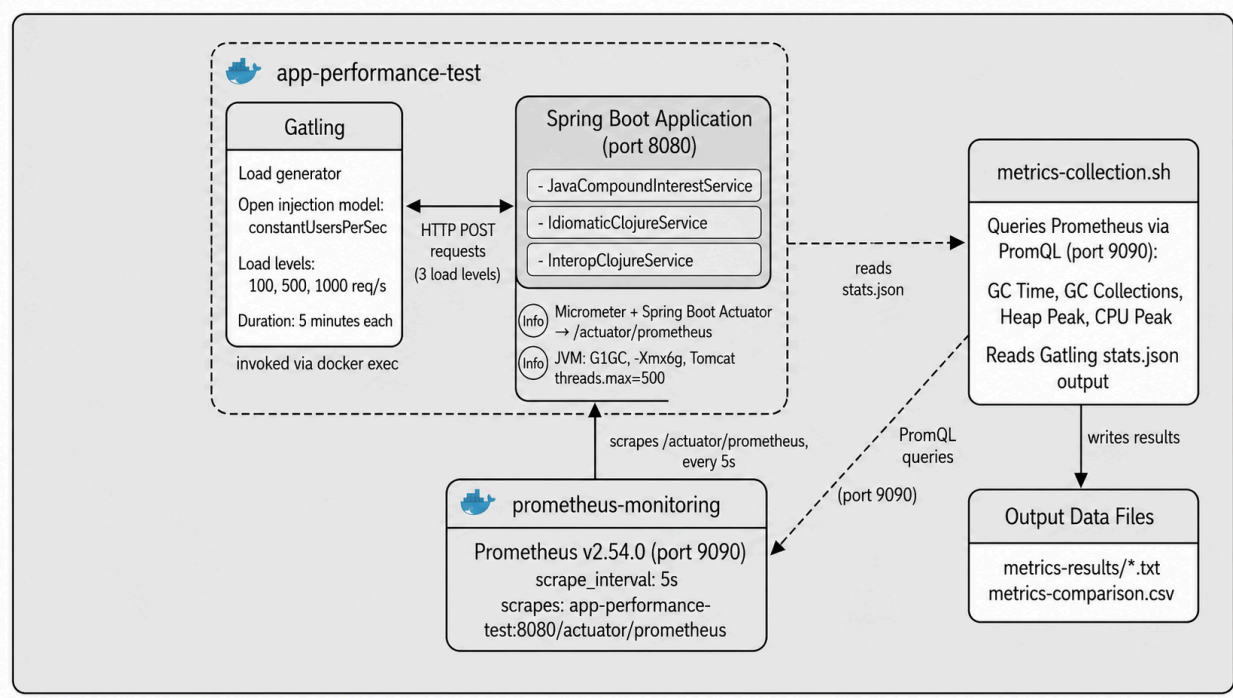


Fonte: elaborado pelo autor (2025).

O diagrama evidencia que as três implementações – Java, Clojure Idiomático e Clojure Interop – operam sob o mesmo contrato de interface *CompoundInterestService*, garantindo que as diferenças observadas nos testes de carga sejam atribuíveis exclusivamente ao paradigma de cada implementação, e não a variações no contrato de entrada e saída ou na camada de exposição HTTP.

Complementarmente, o ambiente de testes é composto por dois containers *Docker*: o container da aplicação *app-performance-test* — que hospeda tanto a aplicação *Spring Boot* quanto o *Gatling*, invocado internamente via `docker exec` — e o container de monitoramento *Prometheus*, *prometheus-monitoring*, interligados por uma rede interna. Um componente adicional, o *script shell metrics-collection.sh*, é executado na máquina *host*, responsável por orquestrar o início e o fim de cada execução, consultar o *Prometheus* via *PromQL* e consolidar as métricas coletadas nos arquivos de saída. Essa arquitetura está representada na Figura 2.

Figura 2 – Diagrama de arquitetura da solução: ambiente de testes de carga.



Fonte: elaborado pelo autor (2026).

O diagrama evidencia que o *Gatling*, embora invocado dentro do mesmo container da aplicação sob teste via `docker exec`, opera como um processo JVM independente, com *heap* e *threads* próprios, distinto do processo da aplicação *Spring Boot*. O container *app-performance-test* dispõe de um orçamento de seis núcleos de CPU, compartilhado entre os dois processos. A métrica de *CPU Peak*, coletada via *Prometheus*, reflete exclusivamente o uso de CPU do processo da aplicação, não capturando o consumo do *Gatling* nesse mesmo orçamento. Ainda assim, o CPU Peak da aplicação manteve-se consistentemente entre 70% e 90% ao longo das execuções, sem atingir saturação, o que constitui evidência indireta — não uma garantia de isolamento completo — de que restava alguma margem de capacidade no orçamento de CPU do container mesmo sob a atividade concorrente do gerador de carga. Essa limitação de instrumentação é discutida na seção 7.3.

4.2. IMPLEMENTAÇÕES E ESTRUTURAS DE DADOS

O algoritmo de juros compostos foi adotado como instrumento de medição, e sua validade metodológica pode ser identificada retrospectivamente a partir de quatro propriedades. O processamento de cada requisição é do tipo *stateless*, ou seja, não há compartilhamento de estado entre requisições, o que elimina contenção de *threads* como variável confundidora da pesquisa e mantém o foco do experimento no custo das abstrações idiomáticas. O algoritmo é também determinístico, no qual o mesmo *input* produz sempre o mesmo *output*, garantindo, dessa forma, reprodutibilidade entre execuções. É parametrizável via variável *years*, permitindo controle preciso da carga computacional por requisição. Por fim, a estrutura do *response body* — um objeto de resultados gerais e um *array* de 100 objetos com resultados anuais — força a alocação iterativa de objetos a cada requisição, expondo diferenças estruturais entre os mecanismos de coleção de cada paradigma, tais como *ArrayList* mutável referente ao código Java e *PersistentVector* imutável referente ao código Clojure Idiomático.

No nível das implementações, a integração entre o ecossistema *Spring Boot* e as variantes Clojure é viabilizada pela classe *ClojureConfiguration*, conforme anotação *Spring @Configuration*, a qual atua como ponte entre os dois paradigmas dentro do contexto da aplicação. No momento da inicialização, o método anotado com *@PostConstruct* carrega os *namespaces* *compound-interest-service-idiomatic* e *compound-interest-service-interop-java* via API *clojure.java.api.clojure*, forçando sua compilação para *bytecode* JVM. Em seguida, a função *create-service* de cada *namespace* é invocada, utilizando método *reify* para instanciar objetos que implementem a *interface CompoundInterestService*. Esses objetos são registrados como *beans Spring* qualificados, tornando-se injetáveis nos *controllers* via anotação *@Qualifier*, logo tornando-se capazes de processar requisições, conforme ilustrado no Código 1.

Código 1 – Integração Spring Boot e Clojure via classe ClojureConfiguration.

```
@Configuration

public class ClojureConfiguration {

    private static final String IDIOMATIC_NAMESPACE =
"com.example.java-clojure-performance-analyzer.service.compound-interest-service-idiomatic";

    private static final String INTEROP_JAVA_NAMESPACE =
"com.example.java-clojure-performance-analyzer.service.compound-interest-service-interop-java";

    private static final String FUNCTION = "create-service";

    private IFn idiomaticServiceFactory;

    private IFn interopJavaServiceFactory;

    @PostConstruct

    public void loadNamespaces() {

        IFn require = Clojure.var("clojure.core", "require");

        require.invoke(Clojure.read(IDIOMATIC_NAMESPACE));

        require.invoke(Clojure.read(INTEROP_JAVA_NAMESPACE));
```

```

        this.idiomaticServiceFactory = Clojure.var(IDIOMATIC_NAMESPACE,
FUNCTION);

        this.interopJavaServiceFactory = Clojure.var(INTEROP_JAVA_NAMESPACE,
FUNCTION);

    }

    @Bean("clojureIdiomaticImplementation")

    public CompoundInterestService compoundInterestService() {

        return (CompoundInterestService) idiomaticServiceFactory.invoke();

    }

    @Bean("clojureInteropJavaImplementation")

    public CompoundInterestService compoundInterestInteropJavaService() {

        return (CompoundInterestService) interopJavaServiceFactory.invoke();

    }

}

```

Fonte: elaborado pelo autor (2025).

As três variantes de microserviço implementam a *interface* *CompoundInterestService* e diferem estruturalmente na escolha da estrutura de dados para construção da coleção de resultados e no mecanismo de iteração adotado. Nesse sentido, a implementação Java imperativa utiliza um *ArrayList* pré-alocado com capacidade inicial igual ao parâmetro *years*, preenchido iterativamente por um laço *for* com mutação de estado a cada ciclo. As operações aritméticas são realizadas diretamente sobre tipos primitivos *double* e *int*, sem encapsulamento em objetos intermediários, conforme ilustrado no Código 2.

Código 2 – Cálculo de juros compostos (camada de serviço): implementação Java imperativa.

```

final double compoundFactor = (1.0 + rate / 100.0);

final List<YearlyInvestmentSummary> yearlyDetailsList = new ArrayList<>(years);

double currentAmount = initialAmount;

for (int year = 1; year <= years; year++) {

    final double startBalance = currentAmount;

```



```

    final double endBalance    = currentAmount * compoundFactor;

    final double interestEarned = endBalance - startBalance;

    yearlyDetailsList.add(new YearlyInvestmentSummary(

        year, startBalance, endBalance, interestEarned));

    currentAmount = endBalance;
}

```

Fonte: elaborado pelo autor (2025).

A implementação Clojure Idiomática substitui o *ArrayList* mutável pela estrutura de dados *PersistentVector* imutável, inicializada vazia e expandindo a cada iteração via método *conj*. O mecanismo de iteração adotado é *loop/recur*, no qual o compilador Clojure otimiza para um salto de volta ao início do laço – semanticamente equivalente ao *goto* no *bytecode* – sem consumo de pilha. A cada chamada do método *conj*, uma nova versão do vetor é criada compartilhando estrutura com a versão anterior, o que resulta em alocação iterativa de objetos intermediários no *heap* a cada requisição (MØLLER; VEILEBORG, 2020), conforme ilustrado no Código 3.

Código 3 – Cálculo de juros compostos (camada de serviço): implementação Clojure Idiomático

```

(let [compound-factor (+ 1.0 (/ rate 100.0))

    yearly-details-list

    (loop [year          1

           current-amount initial-amount

           yearly-details-list []]

    (if (> year years)

        yearly-details-list

        (let [start-balance current-amount

              end-balance    (* current-amount compound-factor)

              interest-earned (- end-balance start-balance)]

            (recur (inc year))

```

```

end-balance

(conj yearly-details-list

  (YearlyInvestmentSummary.

    (int year)

    (double start-balance)

    (double end-balance)

    (double interest-earned)))))))]

```

Fonte: elaborado pelo autor (2025).

A implementação Clojure Interop combina a expressividade sintática do Clojure com estruturas de dados nativas do Java, aproximando-se do seu padrão de alocação da variante imperativa. Um *ArrayList* Java é instanciado com anotações de tipo via *type hint* `^ArrayList`, eliminando a resolução por reflexão em tempo de execução. O incremento do contador de iteração é realizado via *unchecked-inc*, suprimindo as verificações de *overflow* do *runtime* do Clojure, e os valores numéricos são explicitamente coercidos para tipos primitivos *double* e *int*, reduzindo o encapsulamento em objetos *wrapper*. O resultado é um padrão de alocação por mutação direta sem versões intermediárias da coleção, conforme ilustrado no Código 4 (HICKEY, 2020).

Código 4 – Cálculo de juros compostos (camada de serviço): implementação Clojure Interop Java.

```

(let [initial-amount (double (.getInitialAmount request))

      rate            (double (.getAnnualInterestRate request))

      years           (int (.getYears request))

      compound-factor (+ 1.0 (/ rate 100.0))

      ^ArrayList yearly-details-list (ArrayList. years)

      final-amount

      (loop [year          1

             current-amount initial-amount]

```

```

(if (> year years)

  current-amount

  (let [start-balance    current-amount

        end-balance      (* current-amount compound-factor)

        interest-earned  (- end-balance start-balance)]

    (.add yearly-details-list

      (YearlyInvestmentSummary.

        (int year)

        start-balance

        end-balance

        interest-earned))

    (recur (unchecked-inc year)

           end-balance))))]

```

Fonte: elaborado pelo autor (2025).

As três implementações diferem, portanto, no padrão de alocação de objetos produzido a cada requisição. O Java imperativo e o Clojure Interop adotam mutação direta sobre uma única instância da estrutura de dados *ArrayList*, gerando alocação controlada e previsível. Por outro lado, o Clojure Idiomático produz versões intermediárias da estrutura de dados *PersistentVector* a cada iteração – objetos de vida curta que se tornam elegíveis para coleta imediatamente após cada chamada ao método *conj*. Essa distinção entre os paradigmas constitui o mecanismo central investigado nos testes de carga e discutido na seção 7.

4.3. INFRAESTRUTURA DE TESTES

O ambiente de testes foi containerizado via *Docker Compose*, contando com dois containers operando em rede interna isolada, nomeados como *app-performance-test* e *prometheus-monitoring*, reinicializados completamente entre execuções de cada uma das três implementações, com o intuito de eliminar a contaminação de estado entre os ciclos de teste. Com isso, o container da aplicação foi configurado com os seguintes fatores: coletor de lixo *G1GC* via instrução `-XX:+UseG1GC`, sendo o coletor padrão do *OpenJDK* para aplicações de servidor; *heap* inicial de 4GB e máximo de 6GB, `-Xms4g -Xmx6g`, respectivamente; *pause target* de 200ms via `-XX:MaxGCPauseMillis=200`; e limite de memória do container de 8GB via `mem_limit:8g`. Por fim, o servidor web *Tomcat* foi configurado com `thread.max=500`, `max-connections=20000` e `accept-count=1000`, dimensionado para suportar os três níveis de carga sem saturação de conexões como variável confundidora.

Os testes de carga foram executados por meio da ferramenta *Gatling*, biblioteca importada diretamente no arquivo *pom.xml*, utilizando o modelo de injeção aberta via método `injectOpen()` em conjunto com a propriedade `constantUsersPerSec()`, dessa forma, mantendo uma taxa constante de novos usuários virtuais por segundo independentemente do tempo de resposta da API – comportamento representativo de carga real de produção. Ademais, foram aplicados três níveis de carga: 100, 500 e 1000 requisições por segundo, cada execução com duração de 300 segundos, logo totalizando 30.000 requisições no teste inicial com 100 usuários virtuais, 150.000 requisições com 500 usuários virtuais e 300.000 requisições no teste de carga máxima com 1000 usuários virtuais por segundo realizando requisições HTTP na aplicação.

A coleta de métricas da JVM durante tais testes de carga foi realizada via *Prometheus* configurado com `scrape_interval=5s` e `scrape_timeout=4s`, consumindo o *endpoint* `/actuator/prometheus` exposto pelo *Spring Boot Actuator* via biblioteca *Micrometer*. As métricas expostas incluem utilização de CPU, uso de *heap*, contagem e tempo total de pausas de GC e contagem de requisições HTTP processadas.

A integração entre as fontes de dados foi realizada por meio de um *script shell* desenvolvido para automatizar a coleta e consolidação das métricas ao término de cada execução. O *script* consulta o *Prometheus* via *queries PromQL* para extrair as métricas de GC e *heap*, lê o arquivo *stats.json* gerado pelo próprio *Gatling* contendo percentis de latência e *throughput*, e consolida os resultados em dois formatos de saída: um arquivo texto detalhado por execução (*metrics-results/*.txt*) e um arquivo CSV agregado (*metrics-comparison.csv*) utilizado como base para análise estatística conduzida e aprofundada na seção 6. O código-fonte completo do experimento, incluindo as três implementações, os scripts de coleta de métricas e as configurações do ambiente de testes, está disponível publicamente para consulta e replicação (LACERDA, 2026).

4.4. MÉTRICAS COLETADAS - GATLING / PROMETHEUS

As métricas coletadas ao longo dos testes de carga foram obtidas a partir de duas fontes distintas: o *Gatling*, responsável pelas métricas de latência e *throughput* das requisições HTTP, e o *Prometheus*, responsável pelas métricas de comportamento da JVM durante a execução.

A partir do *Gatling*, foram extraídas métricas por execução tais como total de requisições processadas, requisições bem-sucedidas, requisições com falha, taxa de erro percentual; vazão média em requisições por segundo; tempo de resposta mínimo, médio, máximo e desvio padrão em milissegundos; e os percentis de latência: P50, P75, P95 e P99, também em milissegundos. Os percentis P95 e P99 constituem as métricas primárias de análise de tempo de resposta neste trabalho, uma vez que capturam o comportamento da aplicação sob pressão, descrevendo a experiência dos 5% e 1% de requisições com maior latência, respectivamente; portanto, são métricas mais representativas do comportamento sob carga sustentada do que a mediana (DEAN; BARROSO, 2013). Esse comportamento é especialmente relevante em microserviços Java, nos quais pausas do GC são identificadas como propulsoras da latência de cauda em cargas intensivas, tornando os percentis superiores as métricas mais sensíveis para capturar o impacto do coletor de lixo sobre o tempo de resposta (TENNAGE et al., 2019). Essa abordagem de avaliação baseada em quantis de latência é consistente com estudos comparativos de *frameworks* JVM que adotam percentis 0.50, 0.75 e 0.95 como métricas centrais em testes de carga HTTP com *Spring Boot* (WYCIŚLIK et al., 2023).

De forma complementar, o *Prometheus* foi utilizado para extrair métricas via *queries PromQL* tais como tempo total acumulado de pausas do *Garbage Collection* em milissegundos, obtido a partir da métrica `jvm_gc_pause_seconds_sum`; número de ciclos de coleta de lixo, derivado da métrica `jvm_gc_pause_seconds_count`; pico de utilização do *heap* em GB e em percentual relativo ao máximo configurado de 6GB; e pico de utilização de CPU em percentual. Todas as métricas do *Prometheus* foram calculadas como delta entre o valor inicial – coletado antes do início do teste – e o valor final, coletado após a sincronização do scrape ao término da execução, de tal forma a isolar o consumo atribuível exclusivamente ao período de teste.

4.5. PROTOCOLO DE EXECUÇÃO E CONTROLE DE VARIÁVEIS

Inicialmente, vale destaque ao ponto de que todos os testes foram conduzidos em uma máquina com processador Intel Core i5-1335U (13ª geração, 10 núcleos físicos, 12 *threads*, frequência máxima de 4,6GHz), 16GB de memória RAM e armazenamento SSD NVMe de 476,9GB, sob o sistema operacional Ubuntu 24.04.4 LTS. Essa configuração suportava com folga os recursos alocados aos containers *Docker*, dadas as configurações: `cpus:6.0` e `mem_limit:8g`, sem contenção entre máquina *host* e containers durante a execução dos testes.

O experimento compreendeu 54 execuções no total, dada a organização de 3 implementações $\times$ 3 níveis de carga $\times$ 6 repetições. Nesse sentido, os testes foram conduzidos em ordem crescente de carga – 100, 500 e 1000 requisições por segundo – e, dentro de cada nível, em sequência fixa de implementação: Java, Clojure Idiomático e Clojure Interop. Além disso, cada grupo de 6 repetições de uma mesma implementação foi executado de forma contínua antes de avançar para a implementação seguinte.

Para cada execução, o protocolo seguiu as seguintes etapas em sequência: reinicialização completa do ambiente via comando `docker compose down -v`, remoção de volumes e imagens intermediárias; recriação dos containers, garantindo estado limpo de *heap*, GC e métricas do *Prometheus* a cada ciclo. Após a inicialização dos containers, um período de estabilização de 30 segundos foi aguardado, seguido de verificação de saúde da aplicação via comando `curl /actuator/health`, confirmando que a JVM havia completado a inicialização e que o *Spring Boot* estaria pronto para receber requisições antes do início da coleta. Previamente ao disparo do teste, foram verificados o cenário configurado do *Gatling*, o percentual de CPU ociosa da máquina *host* – com alvo acima de 95% – e a disponibilidade de memória RAM – com alvo de 75% – de forma a garantir que processos externos não competiriam por recursos durante a execução. O *script metrics-collection.sh* foi iniciado em um terminal para capturar as métricas iniciais do *Prometheus*, e o teste de carga foi disparado via *script run-gatling.sh* em um segundo terminal. Ao término do *Gatling*, o *script* de captura de métricas foi configurado para aguardar 15 segundos para a devida sincronização da atividade de scrape do *Prometheus*, coletando as métricas finais, lendo o arquivo `stats.json` gerado pelo *Gatling* e consolidando os resultados nos arquivos de saída. Ademais, entre execuções consecutivas foi respeitado um intervalo mínimo de 300 segundos para estabilização do sistema antes do início do ciclo seguinte.

Os testes foram caracterizados como *load testing* com carga sustentada e taxa constante de injeção – modelo distinto de *stress testing*, no qual se busca o ponto de ruptura do sistema, ou de *spike testing*, que aplica picos abruptos. Esse modelo de injeção aberta, viabilizado como uma configuração *Gatling*, é metodologicamente superior a modelos de *loop* fechado para avaliação de latência sob carga sustentada. Em modelos de *loop* fechado, os usuários virtuais são bloqueados aguardando o *response* antes de disparar a próxima requisição HTTP, o que faz com que o gerador de carga reduza automaticamente a taxa de injeção durante períodos de lentidão da aplicação como, por exemplo, durante pausas de GC. Esse comportamento, conhecido como *coordinated omission*, ocorre em praticamente todos os geradores de carga baseados em chamadas síncronas e faz com que o gerador omita exatamente as requisições que ocorreriam durante uma degradação real, produzindo métricas de latência artificialmente otimistas (FRIEDRICH; WINGERATH; RITTER, 2017). Portanto, com o modelo `injectOpen()` em conjunto com a propriedade `constantUsersPerSec`, o *Gatling* continua disparando requisições independentemente do tempo de resposta da aplicação, garantindo que os percentis P95 e P99 capturem a latência real acumulada na fila durante eventos de degradação, como as pausas de GC observadas na variante Java imperativa sob 1000 requisições por segundo. Por fim, o objetivo não foi determinar a capacidade máxima da aplicação, mas observar o comportamento comparativo das três implementações em três regimes de carga progressiva, ou seja, sem pressão de GC com 100 req/s, com pressão moderada a 500 req/s e com pressão suficiente para que diferenças estruturais entre os paradigmas emergissem, com carga final de 1000 req/s.

Após a conclusão da rodada inicial de testes, foi identificado um problema no *script metrics-collection.sh* em que as *queries Prometheus* responsáveis pela coleta de *GC Time* e *GC Collections* utilizavam `result[0]`, capturando apenas uma das séries retornadas pelo *GIGC* em vez da soma de todas. Esse comportamento resultou em subestimação dos valores absolutos dessas métricas na rodada inicial, embora o ordenamento relativo entre implementações tenha sido preservado. A correção – substituição de `result[0]` pela soma de todas as séries *GIGC* – foi implementada antes da execução das rodadas subsequentes, as quais constituem os dados definitivos do estudo. Para os níveis de carga de 100 e 500 requisições por segundo, os dados definitivos são provenientes de uma segunda rodada de 54 execuções conduzida com o *script* corrigido. Para 1000 requisições por segundo, a instabilidade observada nessa segunda rodada, com 4 colapsos em 18 execuções, motivou uma terceira rodada dedicada exclusivamente a esse nível de carga, composta por 7 execuções por implementação, mantendo o mesmo protocolo com reinicialização dos containers entre cada teste.

5. ANÁLISE DE BYTECODE GERADO

Preliminarmente, o *bytecode* JVM consiste no conjunto de instruções de baixo nível produzido pelo compilador a partir do código-fonte de cada implementação e executado pela JVM em tempo de execução. A inspeção dessas instruções permite identificar diferenças estruturais entre os paradigmas que não são visíveis no código-fonte, tais como a forma com que cada implementação realiza operações aritméticas, o modo de gerenciamento de suas estruturas de coleção e qual o volume de despacho dos métodos gerados. Neste trabalho, o *bytecode* foi extraído por meio do comando `javap -c -p`, aplicado diretamente às classes compiladas de cada uma das três implementações, com os resultados consolidados por um *script shell* automatizado que contabilizou a frequência das categorias de instrução relevantes para comparação entre paradigmas.

5.1. BYTECODE JAVA

O *bytecode* gerado pela implementação Java imperativa reflete diretamente as escolhas estruturais descritas na seção 4, como a aritmética sobre tipos primitivos e a mutação direta sobre uma única instância de coleção. Nesse sentido, as operações de cálculo de juros compostos são compiladas para instruções primitivas de ponto flutuante – `dmul`, `dadd` e `dsub` –, sem intermediação de objetos entre os operandos e a unidade aritmética da JVM. O laço `for` é compilado para um `goto` incondicional com a verificação de término via `if_icmpgt`, conforme ilustrado no Código 5.

Código 5 – Excerto de bytecode: implementação Java imperativa (núcleo do laço).

```
53: iload      12      // carrega contador (year)
55: iload      6       // carrega limite (years)
57: if_icmpgt   123     // encerra laço se year > years
60: dload      10      // carrega currentAmount
64: dload      10      // carrega currentAmount (startBalance)
66: dload      7       // carrega compound Factor
68: dmul                // endBalance = currentAmount * compoundFactor
75: dsub                // interestEarned = endBalance - startBalance
117: iinc       12, 1   // incrementa year
120: goto       53      // retorna ao início do laço
```

Fonte: elaborado pelo autor (2025), a partir da extração via `javap -c -p`.

O *ArrayList* é instanciado uma única vez antes do laço, com capacidade inicial igual ao parâmetro `years`, e preenchido por mutação direta a cada iteração via `invokeinterface` sobre a *interface* `List`. Portanto, não há criação de versões intermediárias da coleção ao longo da execução do método, e o padrão resultante é de baixa densidade de indireções, em que as operações determinantes do algoritmo, como multiplicação, subtração e controle de laço, são resolvidas diretamente em nível de instrução JVM, sem despacho para métodos de *runtime* externos.

5.2. BYTECODE CLOJURE IDIOMÁTICO

O *bytecode* gerado pela implementação Clojure Idiomática apresenta padrão estruturalmente distinto da implementação imperativa, uma vez que as operações aritméticas que no Java são compiladas para instruções diretas como `dmul`, `dadd` e `dsub`, são aqui intermediadas pela classe `clojure.lang.Numbers`, despachadas via `invokestatic` a cada operação. Esse comportamento decorre da tipagem dinâmica do Clojure, dado que na ausência de anotações de tipos explícitas, o compilador não pode garantir em tempo de compilação que os operandos são primitivos, delegando a resolução aritmética para o *runtime* (HICKEY, 2020). O núcleo do laço, compilado a partir da construção `loop/recur`, é ilustrado no Código 6.

Código 6 – Excerto de bytecode: implementação Clojure Idiomática (núcleo do laço).

```
17: lload_1           // carrega contador (year)
19: getfield          #17   // carrega limite (years)
22: invokestatic      #35   // Numbers.gt: year > years?
25: ifeq              37    // continua laço se falso
37: aload             5     // carrega current-amount
43: getfield          #19   // carrega compound-factor
47: invokestatic      #39   // Numbers.multiply: currentAmount * compoundFactor
50: dstore            6     // armazena endBalance
54: aload             5     // carrega current-amount (startBalance)
56: invokestatic      #43   // Numbers.minus: endBalance - startBalance
62: invokestatic      #47   // Numbers.inc: incrementa year
37: invokeinterface   #90, 3 // IFn.invoke: conj – expande PersistentVector
46: goto              17    // retorna ao início do laço
```

Fonte: elaborado pelo autor (2025), a partir da extração via `javap -c -p`.

O acumulador da coleção é inicializado a partir de `PersistentVector.EMPTY` e expandido a cada iteração via `conj`, despachado como `invokeinterface` sobre a *interface* `IFn`, sendo assim, a cada chamada uma nova versão do vetor é produzida, compartilhando estrutura de dados com a versão anterior. O resultado é a geração de objetos intermediários de coleção a cada ciclo do laço ocasionando, conseqüentemente, objetos que se tornam imediatamente elegíveis para coleta pelo *Garbage Collector* após cada iteração. Conclui-se, portanto, que o padrão geral da implementação idiomática é de alta densidade de chamadas ao *runtime* Clojure e de alocações intermediárias por iteração, em contraste direto com o padrão de mutação direta observado na implementação Java.

5.3. BYTECODE CLOJURE INTEROP

O bytecode gerado pela implementação Clojure Interop aproxima-se estruturalmente da implementação imperativa Java em dois aspectos determinantes: a aritmética sobre tipos primitivos e o padrão de mutação direta sobre uma única instância de coleção. O *type hint* `^ArrayList` no código-fonte elimina a resolução por reflexão em *runtime*, ou seja, o compilador Clojure emite `checkcast` diretamente para a classe `java.util.ArrayList`, resolvendo a tipagem da variável em tempo de compilação. Ademais, o incremento do contador via `unchecked-inc` é compilado para `ladd` sobre `long`, suprimindo a verificação de *overflow* do *runtime* Clojure presente na implementação idiomática. O núcleo do laço é ilustrado no Código 7.

Código 7 – Excerto de bytecode: implementação Clojure interop (núcleo do laço).

```
7: lload_1                // carrega contador (year)
9: getfield                #23    // carrega limite (years)
13: lcmp                   // compara year com years
14: ifle                   25      // continua laço se year <= years
29: getfield                #21    // carrega compound-factor
33: dmul                   // endBalance = currentAmount * compoundFactor
40: dsub                   // interestEarned = endBalance - startBalance
47: checkcast              #35     // verifica tipo: ArrayList
```

```

91: invokevirtual #55          // ArrayList.add: adiciona resultado à coleção
96: lconst_1
97: ladd                        // incrementa year (unchecked-inc)
02: goto          7            // retorna ao início do laço

```

Fonte: elaborado pelo autor (2025), a partir da extração via `javap -c -p`.

O *ArrayList* é preenchido por `invokevirtual` diretamente sobre a classe concreta `java.util.ArrayList`, em contraste com o `invokeinterface` sobre `IFn` utilizado pelo Clojure Idiomático para despachar `conj`. Nesse contexto, não há criação de versões intermediárias da coleção, uma vez que a mutação ocorre diretamente sobre a única instância alocada antes do laço. O padrão geral da implementação Interop é, portanto, de aproximação ao perfil Java em termos de alocação de coleção, mantendo a infraestrutura de runtime Clojure nas operações de conversão de tipos, como as chamadas a `RT.intCast` e `Double.valueOf` visíveis nas linhas 55 e 56 do excerto completo.

5.4. COMPARATIVO DE INSTRUÇÕES

A Tabela 1 consolida as contagens das categorias de instrução mais relevantes extraídas pelo comando `javap` para as três implementações, permitindo a comparação direta dos padrões de *bytecode* produzidos por cada paradigma.

Tabela 1 – Comparativo de instruções bytecode por implementação.

Instrução	Java	Clojure Idiomático	Clojure Interop
<code>invokestatic</code>	7	56	50
<code>invokevirtual</code>	6	14	11
<code>invokeinterface</code>	1	7	4
<code>new (alocações)</code>	4	8	9
Operações primitivas double	5	1	4

Fonte: elaborado pelo autor (2025), a partir da extração via `javap -c -p`.

A diferença mais expressiva entre os três perfis reside na densidade de `invokestatic`, em que o Clojure Idiomático produz 56 ocorrências contra 7 do Java, reflexo direto da intermediação aritmética por `clojure.lang.Numbers` descrita na subseção 5.2. Por outro lado, o Clojure Interop reduz parcialmente esse volume para 50, mas mantém densidade substancialmente superior a Java em razão da infraestrutura de *runtime* Clojure presente nas demais operações. Em sentido inverso, o Java concentra 5 operações aritméticas primitivas sobre `double`, enquanto Clojure Idiomático apresenta apenas 1, logo evidenciando que a aritmética é quase integralmente absorvida pelo *runtime* Clojure nessa implementação. Por sua vez, Clojure Interop recupera parcialmente esse padrão com 4 operações primitivas, aproximando-se de Java nas instruções aritméticas determinantes do algoritmo.

6. RESULTADOS DOS TESTES DE CARGA

Esta seção apresenta os resultados quantitativos organizados em quatro subseções: análise de throughput (6.1), análise de latência (6.2), análise de *Garbage Collection* (6.3) e validação estatística e reprodutibilidade (6.4). Os resultados são baseados em três rodadas independentes de testes, sendo uma rodada inicial de 54 execuções, cujas *queries Prometheus* para dados de GC apresentavam problema de coleta, como descrito na seção 4.5, com dados de 100 e 500 req/s mantidos como referência histórica e limitação documentada; uma segunda rodada de 54 execuções conduzida com *script* corrigido, a qual constitui os dados definitivos para 100 e 500 requisições por segundo, e uma terceira rodada de 21 execuções dedicada exclusivamente a 1000 requisições por segundo, constituindo os dados definitivos para esse nível de carga. Para cada grupo, os valores reportados correspondem às médias aritméticas das execuções válidas por implementação e nível de carga, calculadas a partir dos dados coletados conforme descrito na seção 4. No total, seis execuções foram excluídas por invalidez qualitativa – todas as 1000 req/s, por apresentarem taxa de erro superior a zero, critério adotado como condição suficiente de descarte pelo protocolo – e sete execuções adicionais foram identificadas como *outliers* estatísticos pelo critério *z-score* ($|z| > 2$) aplicado sobre a métrica P95 (GRUBBS, 1969; GEORGES et al., 2007), sendo três na rodada inicial, quatro na segunda rodada e nenhum na terceira, com análise de sensibilidade documentada na seção 6.4. Por fim, os resultados são apresentados de forma descritiva e a interpretação causal dos padrões observados, em especial a relação entre comportamento de GC e estabilidade sob saturação, será tratada na seção 7.

6.1. ANÁLISE DE THROUGHPUT

Inicialmente, as três implementações atingiram o *throughput* alvo em todos os níveis de carga: 100, 500 e 1000 requisições por segundo, com taxa de erro de 0% nas execuções válidas. Esse resultado confirma que o modelo de injeção de carga aberta configurado na ferramenta *Gatling* operou conforme especificado, sem degradação de *throughput* decorrente das implementações em si. Todavia, exceções foram registradas exclusivamente a 1000 req/s, nível de carga em que seis execuções foram excluídas por invalidez qualitativa nas três rodadas independentes, conforme consolidado na Tabela 2. Em todas essas execuções, a exclusão decorreu de taxa de erro superior a zero, critério adotado por incompatibilidade com o estado estável exigido pelo protocolo, e a assinatura de *GC thrashing* registrada nas demais execuções constitui indicador adicional de degradação, não condição necessária para o descarte – GC Time entre 9.527ms e 9.631ms, GC Collections entre 49 e 54 ciclos e heap colapsado entre 0,75GB e 1,2GB, anormalmente inferior ao pico normal de aproximadamente 2,33GB. O Java é a única implementação a apresentar colapso nas três rodadas independentes. A equivalência de *throughput* nas execuções válidas estabelece a comparabilidade das análises subsequentes, uma vez que as diferenças observadas nas seções 6.2 e 6.3 refletem variações de latência e comportamento de coleta de lixo, e não diferenças no volume de requisições efetivamente processadas.

Tabela 2 – Execuções excluídas por invalidez qualitativa: taxa de erro e perfil de falha HTTP.

Implementação	Rodada	Taxa de Erro	Timeout resposta (genérico)	Timeout resposta (IPv4)	Timeout resposta (IPv6)	Timeout conexão TCP (10s)	Fechamento abrupto
Java	V1	9,70%	46,45%	22,30%	2,58%	20,79%	7,89%
Java	V2	18,60%	53,56%	25,43%	5,15%	11,14%	4,72%
Clojure Idiomático	V2	0,60%	-	47,14%	4,28%	29,21%	19,37%
Clojure Idiomático	V2	20,90%	45,96%	23,86%	7,21%	15,50%	7,47%
Clojure Interop	V2	24,70%	45,29%	18,75%	8,92%	19,15%	7,89%
Java	V3	25,10%	59,14%	25,66%	5,25%	6,53%	3,42%

Fonte: elaborado pelo autor, a partir dos relatórios HTML gerados pelo Gatling (2026).

A Tabela 2 detalha, além da taxa de erro agregada, o tipo de falha HTTP registrado pelo *Gatling* em cada uma das seis execuções excluídas por invalidade qualitativa. Três categorias de erro se repetem: *timeout* de resposta (*Request timeout after 60000ms*), quando a requisição estabelece conexão mas não recebe resposta dentro do limite de 60 segundos configurado no próprio *Gatling*; *timeout* de conexão TCP (*ConnectTimeoutException: connection timed out after 10000ms*), quando a conexão não é estabelecida dentro do limite de 10 segundos; e fechamento abrupto de conexão (*IOException: Premature close*), quando o *socket* é encerrado no meio da troca HTTP. As três variantes de *timeout* de resposta reportadas na Tabela 2 (genérico, IPv4 e IPv6) correspondem ao mesmo tipo de falha, diferindo apenas no endereço de *loopback* resolvido pela JVM em cada tentativa; somadas, essa categoria responde por entre 51,42% das falhas, na execução Clojure Idiomático com 0,60% de erro, e 90,05%, na execução Java da terceira rodada com 25,10% de erro — sendo, em todas as seis execuções, a categoria predominante, seguida pelo *timeout* de conexão TCP e, por fim, pelo fechamento abrupto.

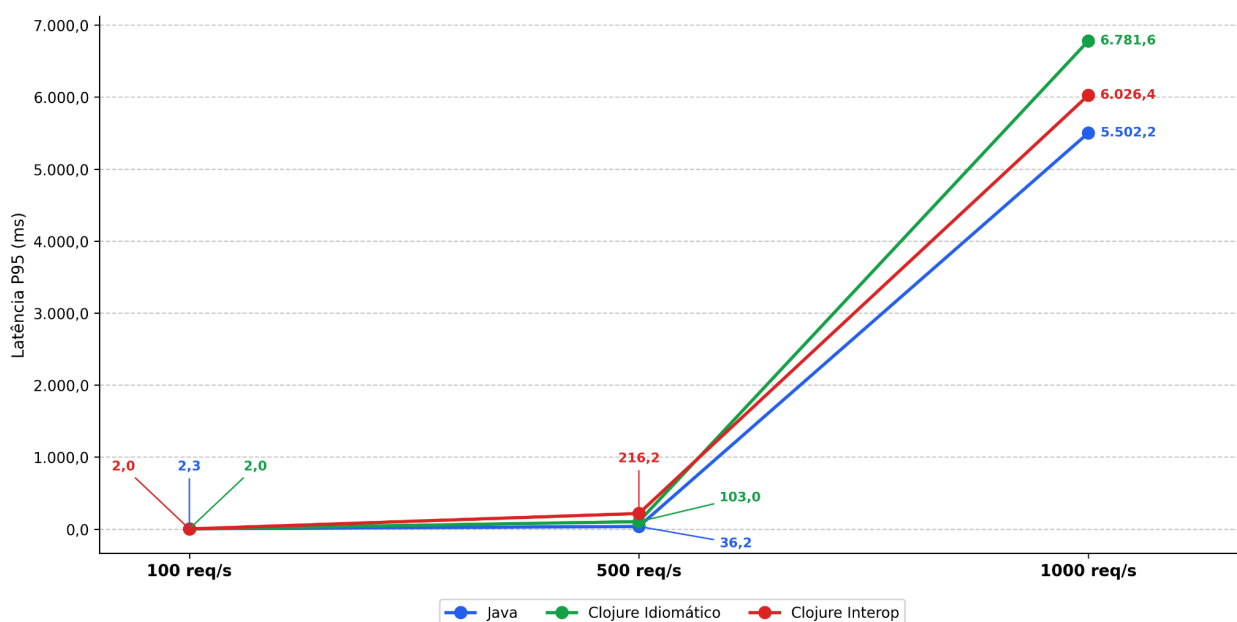
6.2. ANÁLISE DE LATÊNCIA

A análise de latência considera os percentis P95 e P99 como métricas primárias, uma vez que capturam o comportamento relevante da cauda da distribuição de *response time* sob carga sustentada (DEAN; BARROSO, 2013). Os valores reportados correspondem às médias das execuções válidas por grupo, conforme descrito na introdução desta seção.

De acordo com o teste de carga promovido com uma taxa de 100 requisições por segundo, as três implementações apresentaram latência P95 equivalente – Java 2,33ms, Clojure Idiomático 2ms e Clojure Interop 2ms –, sem diferença operacionalmente relevante entre os paradigmas nesse nível de carga. O comportamento de P99 foi igualmente uniforme, com Java registrando 3,83ms, Clojure Idiomático 4ms e Clojure Interop 3,8ms. Esses resultados indicam que, sob baixa carga, o custo das abstrações funcionais não se manifesta em degradação de latência observável.

Com uma carga de 500 requisições por segundo, o comportamento divergiu entre as implementações: o Java registrou o menor P95 de 36,2ms, seguido pelo Clojure Idiomático com 103ms e pelo Clojure Interop com 216,2ms. Em relação ao percentil P99, as três implementações convergiram para valores próximos – Java 2.145,6ms, Clojure Idiomático 2.049ms e Clojure Interop 2.097ms –, sem diferença operacionalmente relevante entre os paradigmas nesse percentil. A Figura 3 ilustra a progressão dos valores de P95 entre os três níveis de carga.

Figura 3 – Latência P95 por implementação e nível de carga (médias em ms).

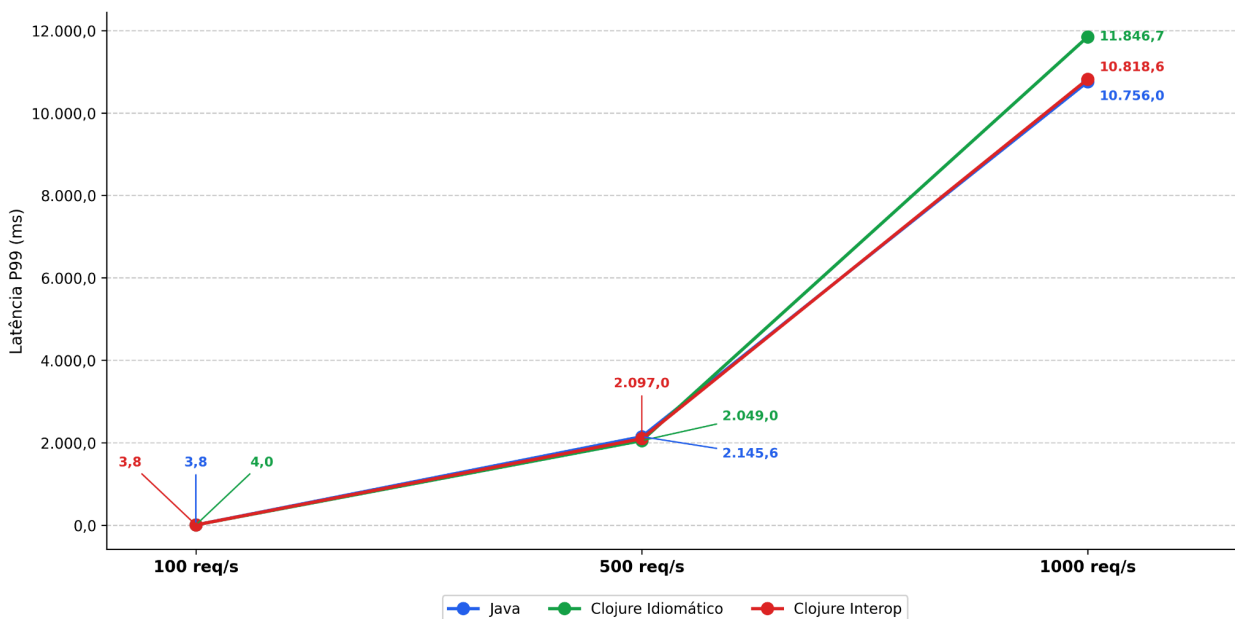


Fonte: elaborado pelo autor (2026).

O gráfico revela dois padrões distintos: a 100 req/s as três implementações são indistinguíveis em P95; a 500 req/s emerge uma separação com vantagem do Java; e a 1000 req/s todas as implementações sofrem elevação abrupta de latência, com o Clojure Idiomático registrando o maior P95 médio – padrão que, no entanto, não captura a dispersão inter-execuções estruturalmente diferente entre os paradigmas, quantificada pelo cálculo de Coeficiente de Variação na seção 6.4.

A 1000 requisições por segundo, as latências médias das execuções estáveis convergiram entre as três implementações: Java registrou P95 de 5.502,2ms e P99 de 10.756ms, Clojure Idiômático P95 de 6.781,57ms e P99 de 11.846,71ms, e Clojure Interop P95 de 6.026,43ms e P99 de 10.818,57ms. A Figura 4 detalha os valores de P99 por implementação e nível de carga. Embora as médias de latência sejam comparáveis entre as implementações nas execuções estáveis, o Java apresentou CV% de 31,14% para P95 contra 13,05% do Clojure Idiômático e 15,73% do Clojure Interop, evidenciando instabilidade diferencial significativa sob saturação. Esse achado – convergência de latência nas execuções estáveis combinada à maior variabilidade do Java sob carga extrema –, constitui o resultado central do presente estudo e será interpretado causalmente na seção 7.

Figura 4 – Latência P99 por implementação e nível de carga (médias em ms).



Fonte: elaborado pelo autor (2026).

Em contraste com o P95, o P99 revela convergência entre as três implementações já a 500 req/s – os valores ficam entre 2.049ms e 2.145,6ms –, indicando que eventos de GC com duração suficiente para inflar o percentil 99 ocorrem com frequência e magnitude semelhantes nas três implementações nesse regime, independentemente do paradigma. A 1000 req/s, as três implementações permanecem na mesma ordem de grandeza em P99 – entre 10.756ms e 11.846,71ms –, reforçando que o achado central do estudo não está na média dos percentis de cauda, mas na variância inter-execuções do Java, quantificada pelo CV% de P95 na seção 6.4.

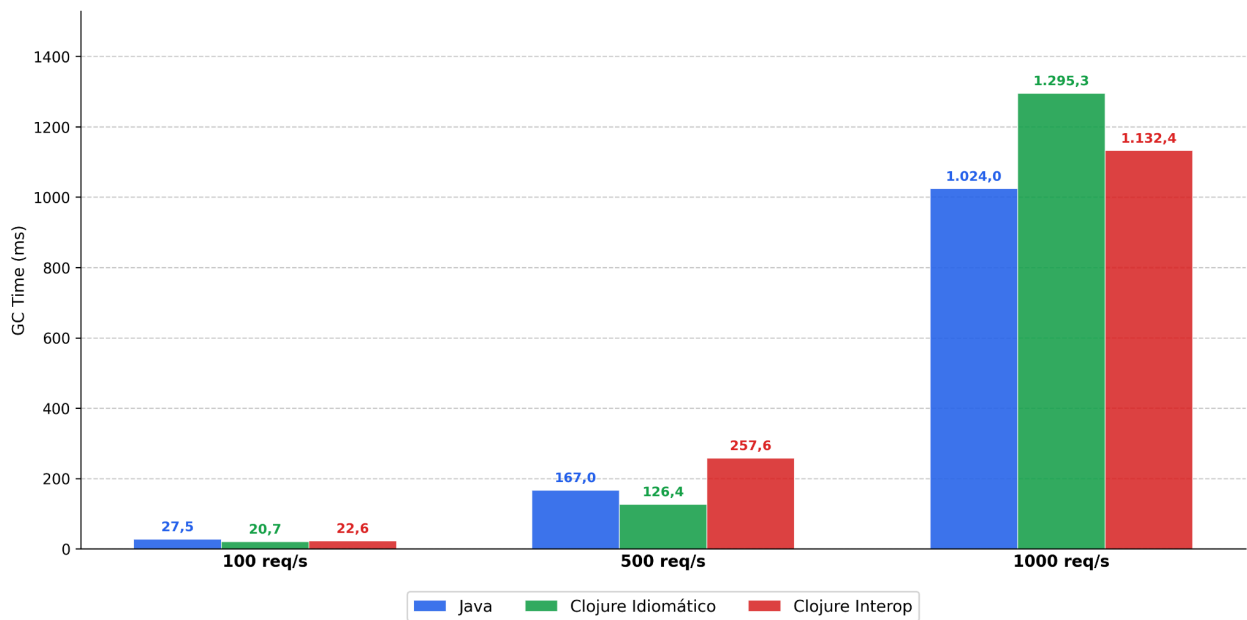
6.3. ANÁLISE DE GARBAGE COLLECTION

A análise da atividade de coleta de lixo promovida pelo componente *Garbage Collector* considera o tempo total de coleta – *GC Time* – e o número total de coletas realizadas – *GC Collections* – como métricas primárias, uma vez que permitem caracterizar o comportamento do coletor *G1GC* em cada implementação sob carga crescente. Os valores reportados correspondem às médias das execuções válidas por grupo.

Sob uma taxa de 100 requisições por segundo, o Clojure Idiomático registrou o menor *GC Time* de 20,67ms, seguido pelo Clojure Interop com 22,6ms e pelo Java com 27,5ms. Para a métrica de *GC Collections*, o Clojure Idiomático e o Clojure Interop realizaram em média 1,33 e 1,4 coletas, respectivamente, contra 1,67 do Java. Dados os valores obtidos, conclui-se que, sob baixa carga, o Clojure Idiomático apresentou menor pressão sobre o coletor de lixo na JVM em ambas as métricas, embora as diferenças absolutas entre as três implementações sejam de pequena magnitude nesse regime.

A 500 requisições por segundo, o Clojure Idiomático registrou o menor *GC Time* de 126,4ms, seguido pelo Java com 167ms e pelo Clojure Interop com 257,6ms. Para a métrica *GC Collections*, as três implementações apresentaram valores próximos, sendo Java 5,6 coletas, Clojure Idiomático 6,4 e Clojure Interop 5,8, sem diferença operacionalmente relevante entre os grupos nessa métrica.

Figura 5 – GC Time por implementação e nível de carga (médias das execuções válidas em ms).

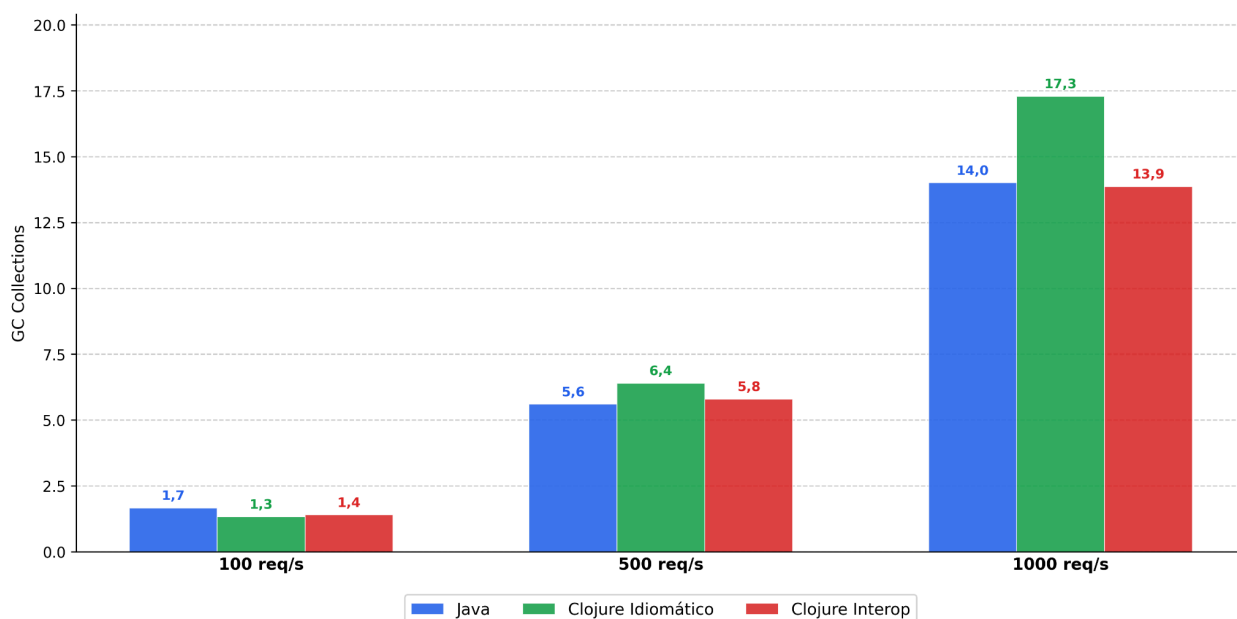


Fonte: elaborado pelo autor (2026).

A Figura 5 evidencia que o Clojure Idiômático mantém o menor *GC Time* a 100 e 500 req/s, invertendo a posição apenas a 1000 req/s, quando o Java passa a registrar o menor tempo acumulado de coleta. Essa inversão é interpretada com cautela: as execuções Java descartadas por invalidez qualitativa a 1000 req/s – aquelas com assinatura de *GC thrashing* documentada na seção 6.1 – concentravam os maiores tempos de coleta, de forma que o *GC Time* médio das execuções válidas remanescentes subestima o custo real do coletor Java nesse regime. A leitura do *GC Time* médio isoladamente, sem considerar o CV% de P95 e os descartes, produziria uma imagem inversa à realidade estrutural observada.

A 1000 requisições por segundo, o Java registrou o menor *GC Time* de 1.024ms, seguido pelo Clojure Interop com 1.132,43ms e pelo Clojure Idiomático com 1.295,29ms. Em relação à métrica de *GC Collections*, o Clojure Idiomático realizou o maior número de coletas, sendo 17,29 em média contra 14 do Java e 13,9 do Clojure Interop. A Figura 6 apresenta os valores de *GC Collections* por implementação e nível de carga. Esses dados confirmam que o Clojure Idiomático gera consistentemente maior pressão sobre o coletor de lixo do que as demais implementações em todos os níveis de carga – comportamento coerente com a densidade de 56 ocorrências de *invokestatic* por requisição identificada na análise de *bytecode* –, embora essa pressão adicional não se traduza em penalidade de latência nas execuções estáveis, conforme interpretado causalmente na seção 7.

Figura 6 – *GC Collections* por implementação e nível de carga (médias das execuções válidas).



Fonte: elaborado pelo autor (2026).

O gráfico evidencia que o Clojure Idiomático passa a realizar consistentemente mais coletas do que as demais implementações a partir de 500 req/s – padrão que se intensifica a 1000 req/s, com 17,29 coletas em média contra 14 do Java e 13,9 do Clojure Interop –, resultado direto da alocação iterativa de versões do *PersistentVector* a cada requisição. A leitura conjunta das Figuras 5 e 6 revela o mecanismo central: mais coletas com menor custo individual por coleta no Clojure Idiomático, contra menos coletas com maior risco de escalada para regime de *Mixed* ou *Full GC* no Java sob saturação.

6.4. VALIDAÇÃO ESTATÍSTICA E ANÁLISE DE REPRODUTIBILIDADE

A reprodutibilidade dos resultados foi avaliada pelo *Coeficiente de Variação* inter-execuções calculado para cada grupo de implementação e nível de carga, após a exclusão da execução inválida e dos *outliers* estatísticos identificados pelo critério *z-score* ($|z| > 2$) (GRUBBS, 1969; GEORGES et al., 2007). O CV% é adotado como métrica de reprodutibilidade por ser adimensional, permitindo, dessa forma, comparação direta entre grupos com magnitudes distintas (JAIN, 1991; HOEFLER; BELLI, 2015).

Seis execuções foram excluídas por invalidez qualitativa ao longo das três rodadas de testes, todas a 1000 requisições por segundo, por apresentarem taxa de erro incompatível com estado estável combinada à assinatura de GC *thrashing*, conforme detalhada na seção 6.1. Essas exclusões são anteriores e independentes da análise de *outliers* por *z-score*.

Sete execuções foram identificadas como *outliers* estatísticos pelo critério *z-score* ($|z| > 2$), distribuídas entre as três rodadas, sendo três na rodada inicial, concentradas nas métricas de latência e GC a 500 req/s do Clojure Idiomático e a 500 e 1000 req/s do Clojure Interop, e quatro na segunda rodada, concentradas nas métricas de latência a 500 req/s das três implementações. A Tabela 3 apresenta a análise de sensibilidade comparando as médias com e sem exclusão desses *outliers*.

Tabela 3 – Médias com e sem exclusão de outliers por implementação e nível de carga.

IMPL	Nível de Carga	P95 com outliers (ms)	P95 sem outliers (ms)	P99 com outliers (ms)	P99 sem outliers (ms)	GC Time com outliers (ms)	GC Time sem outliers (ms)
Java	100 req/s	2,33	2,33	3,83	3,83	27,5	27,5
	500 req/s	146,67	36,2	2.440,5	2.145,6	217,33	167
	1000 req/s	24.765,43	5.502,2	33.187	10.756	2.120,14	1.024
Clojure Idiomático	100 req/s	20	20	4	4	20,67	20,67
	500 req/s	436,5	103	2.646,67	2.049	184,33	126,4
	1000 req/s	6.781,57	6.781,57	11.846,71	11.846,71	1.295,29	1.295,29
Clojure Interop	100 req/s	2,17	20	3,83	3,8	21,5	22,6
	500 req/s	4.483,67	216,2	6.789,5	2.097	360,5	257,6
	1000 req/s	6.026,43	6.026,43	10.818,57	10.818,57	1.132,43	1.132,43

Fonte: elaborado pelo autor (2026).

A ausência de diferença relevante entre as colunas confirma que os outliers não eram observações influentes, ou seja, sua exclusão não altera as conclusões do estudo.

Os valores de *Coeficiente de Variação* obtidos no conjunto inter-execuções, e calculados após a exclusão das execuções inválidas e dos outliers estatísticos – consolidados na Tabela 4 –, apresentam comportamento heterogêneo entre métricas e níveis de carga, revelando padrões estruturalmente distintos entre as implementações. A 100 req/s, os CV% de latência são baixos nas três implementações: abaixo de 23% em P95 e de 12% em P99, confirmando que as médias reportadas são reprodutíveis e que o regime de baixa carga não expõe diferenças estruturais entre os paradigmas.

A 500 req/s, os CV% de P95 permanecem elevados mesmo após a exclusão dos outliers – 117,21% para o Java, 124,63% para o Clojure Idiomático e 129,69% para o Clojure Interop –, valores superiores a 100% que indicam que o desvio padrão supera a própria média em todas as implementações. Essa magnitude é estruturalmente explicável pela dispersão intrínseca das execuções válidas nesse regime intermediário: em todas as três implementações, execuções individuais transitam entre latências baixas e latências elevadas sem um padrão estável definido, o que é coerente com um nível de carga em que a pressão de GC ainda não é suficientemente uniforme para determinar o comportamento de cauda de forma reprodutível. Por essa razão, a interpretação do CV% a 500 req/s é qualitativa – evidência de instabilidade nesse regime –, e não comparativa entre implementações, dado que as três apresentam dispersão de magnitude semelhante. A redução dos CV% em relação aos valores pré-exclusão confirma que a remoção dos outliers foi estatisticamente justificada.

A 1000 req/s, o padrão de CV% é o mais revelador do estudo: o Java apresenta CV% de P95 de 31,14% contra 13,05% do Clojure Idiomático e 15,73% do Clojure Interop. Essa assimetria não reflete instabilidade experimental, dado que o protocolo e o ambiente de testes são idênticos para as três implementações, mas sim a instabilidade estrutural diferencial do Java sob saturação: execuções estáveis produzem latências convergentes com as demais implementações, enquanto execuções próximas ao limiar de colapso de GC produzem latências significativamente superiores, inflando a dispersão inter-execuções.

As métricas de *GC Time* e *GC Collections* apresentaram CV% mais elevados em determinadas implementações, comportamento esperado dado o caráter não-determinístico do coletor *G1GC*, cujas pausas variam em duração e frequência entre execuções independentes (GEORGES et al., 2007; KALIBERA; JONES, 2013). Por fim, a variabilidade observada nas métricas de GC não compromete as conclusões do estudo, uma vez que o resultado central – a instabilidade diferencial do Java sob saturação – está sustentado pelo CV% de P95, cuja diferença entre Java e Clojure Idiomático – 31,14% contra 13,05%, respectivamente – é de magnitude suficiente para sustentar a comparação mesmo na presença de dispersão inter-execuções nas métricas de GC.

Tabela 4 – CV% inter-execuções (após exclusão de outliers).

IMPL	Nível de Carga	P95 CV%	P99 CV%	GC Time CV%	GC Collections CV%
Java	100 req/s	22,13	10,65	38,88	30,98
	500 req/s	117,21	39,1	64,12	15,97
	1000 req/s	31,14	12,74	16,23	0
Clojure Idiomático	100 req/s	0	0	60,25	61,24
	500 req/s	124,63	40,33	17,18	8,56
	1000 req/s	13,05	13,42	23,66	9,28
Clojure Interop	100 req/s	0	11,77	68,16	63,89
	500 req/s	129,69	26,8	64,51	7,71
	1000 req/s	15,73	9,43	12,16	6,49

Fonte: elaborado pelo autor (2026).

7. DISCUSSÃO

7.1. CORRELAÇÃO BYTECODE-PERFORMANCE

A análise comparativa do *bytecode* gerado pelas três implementações, conduzida na seção 5, identificou diferenças estruturais que se manifestam de forma distinta em cada nível de carga investigado. A correlação entre esses padrões de instrução e os *perfis de performance* observados na seção 6 constitui o eixo interpretativo central deste trabalho, cuja cadeia causal pode ser enunciada como: paradigma $\square$ bytecode $\square$ padrão de alocação no *heap* $\square$ comportamento do GC $\square$ latência observada.

A 100 req/s, as três implementações apresentaram latência P95 equivalente: Java 2,33ms, Clojure Idiomático 2ms e Clojure Interop 2ms, sem diferença operacionalmente relevante. Nesse regime, a pressão sobre o coletor de lixo é baixa o suficiente para que as diferenças estruturais de *bytecode* não se traduzam em degradação observável no tempo de resposta das requisições. Ainda assim, o comportamento do GC já revela a distinção paradigmática, uma vez que o Clojure Idiomático registrou o menor *GC Time* de 20,67ms e 1,33 coletas em média, contra 27,5ms e 1,67 coletas do Java. Esse resultado é coerente com a alocação iterativa de versões intermediárias da estrutura de dados *PersistentVector* a cada requisição, conforme documentado na análise de *bytecode* pela densidade de 56 ocorrências de `invokestatic` contra apenas 7 do Java, cujos objetos tornam-se elegíveis para coleta imediatamente após cada chamada ao método `conj`. Esses objetos de vida exígua alinham-se ao pressuposto fundamental sobre o qual coletores de lixo geracionais modernos foram projetados – princípio observado por Lieberman e Hewitt (1983) e Ungar (1984) e denominado *weak generational hypothesis* (hipótese geracional fraca) na literatura de coletores de lixo (ZHAO; BLACKBURN, 2020) –, segundo o qual objetos recém-alocados têm maior probabilidade de se tornarem lixo do que objetos mais antigos, favorecendo coletas cirúrgicas na *Young Generation* com menor custo acumulado por requisição.

A 500 req/s, o comportamento divergiu de forma acentuada. O Java registrou o menor P95 de 36,2ms, seguido pelo Clojure Idiomático com 103ms e pelo Clojure Interop com 216,2ms. Esse padrão é causalmente explicado pelo *bytecode*: o Java, ao operar com aritmética primitiva direta e mutação sobre uma única instância de *ArrayList*, não gera versões intermediária da coleção a cada iteração, mantendo pressão de GC inferior nesse regime intermediário. O Clojure Interop, embora aproxime seu padrão de alocação do Java via *type hints* e *unchecked-inc*, mantém infraestrutura de *runtime* Clojure nas demais operações, o que se reflete nas 50 ocorrências de *invokestatic* e resulta pressão de GC superior ao Java com latência de cauda mais elevada. O Clojure Idiomático, por sua vez, mantém sua alocação iterativa de estruturas *PersistentVector*, gerando pressão crescente sobre o coletor à medida que o volume de requisições simultâneas aumenta. O *GC Time* a 500 req/s confirma essa leitura: Clojure Idiomático acumulou 126,4ms de pausas contra 167ms do Java e 257,6ms do Clojure Interop, com 6,4 coletas contra 5,6 e 5,8, respectivamente. Embora o Clojure Idiomático apresente menor *GC Time* que o Java a 500 req/s, essa vantagem não se traduz em menor latência de cauda nesse regime, dado que a alocação iterativa de objetos intermediários ainda pressiona o coletor de forma suficiente para comprometer o P95.

A 1000 req/s, as latências médias das execuções estáveis convergiram entre as três implementações: Java P95 de 5.502,2ms, Clojure Idiomático 6.781,57ms e Clojure Interop 6.026,4ms, mas o Java apresentou CV% de P95 de 31,14% contra 13,05% do Clojure Idiomático e 15,73% do Clojure Interop, evidenciando instabilidade diferencial significativa sob saturação. A hipótese explanatória central sustenta que essa instabilidade é mediada pelo comportamento do coletor *G1GC* sob pressão geracional. O Clojure Idiomático, ao alocar objetos de vida curta descartados imediatamente após cada chamada ao método *conj*, favorece o padrão sobre o qual o *G1GC* foi projetado: a taxa de sobrevivência desses objetos ao *Minor GC* é mínima, de modo que a fase de evacuação encontra poucos objetos vivos para copiar, logo mantendo pausas curtas e cirúrgicas, conforme documentado por Zhao e Blackburn (2020), os quais demonstram que a coleta geracional do *G1GC* reduz as pausas de P95 em até 93% em relação a varreduras não geracionais.

O Java, por outro lado, apresenta um modo de falha estruturalmente distinto sob carga sustentada a 1000 req/s, uma vez que a estrutura de dados *ArrayList* mutável, ao ser modificado iterativamente ao longo do processamento de cada requisição, permanece alcançável na pilha do *thread* durante toda a duração do processamento. Sob alta taxa de injeção, o intervalo entre *Minor GCs* diminui enquanto o tempo de processamento por requisição se mantém, logo criando uma janela de sobreposição na qual o coletor dispara atividade com centenas de requisições ainda em processamento. Os objetos intermediários vinculados a essas requisições em voo são identificados como vivos pela varredura de raízes e evacuados para a *Survivor Space* – pressão suficiente para que o mecanismo de promoção adaptativa do *G1GC* reduza o limiar de *tenuring* e promova esses objetos prematuramente para a *Old Generation* (DETLEFS et al., 2004). Uma vez na *Old Generation*, a mutabilidade do *ArrayList* torna-se um passivo arquitetural: modificações subsequentes sobre estruturas promovidas geram referências cruzadas entre gerações: *Old-to-Young*, em que o *G1GC* deve rastrear via *Post-Write Barriers* e *Remembered Sets*. Esse rastreamento impõe custo contínuo sobre as *threads* mutadoras e custo adicional durante as pausas *STW*, conforme documentado por Zhao e Blackburn (2020), que quantificam a sobrecarga das barreiras em aproximadamente 12% do desempenho total do sistema. O acúmulo de objetos na *Old Generation* força a transição para *Mixed GCs* ou *Full GCs* com varredura *STW* de todo o *heap*, cujas pausas escalam linearmente com a capacidade configurada de 4 a 6GB.

O dado empírico mais revelador para essa interpretação é a combinação entre número de coletas e *GC Time* a 1000 req/s: o Clojure Idiomático realizou o maior número de coletas – 17,29 em média contra 14 do Java e 13,9 do Clojure Interop –, mas com *GC Time* acumulado de 1.295,3ms contra 1.024ms do Java. Isso implica que o custo médio por coleta do Java foi de aproximadamente 73ms, contra cerca de 75ms do Clojure Idiomático, valores próximos em execuções estáveis, mas que nas execuções em que o Java colapsou, o custo por coleta escalou de forma não linear, produzindo a dispersão inter-execuções documentada pelo CV% de 31,14%. Essa assimetria é explicada pelo custo diferencial entre *Minor GCs* operando majoritariamente sobre a *Young Generation* e coletas mistas ou completas sobre a *Old Generation*, identificadas como fatores dominantes na duração das pausas de GC em coletores estilo G1 (DETLEFS et al., 2004). O comportamento do *Garbage Collector*, portanto, não apenas mensura a diferença entre os paradigmas, mas medeia, amplificando sob alta carga as consequências estruturais de cada escolha de alocação.

7.2. TRADE-OFFS IDENTIFICADOS

Os dados apresentados na seção 6 e interpretados causalmente em 7.1 revelam que nenhuma das três implementações domina as demais em todos os regimes de carga investigados, ou seja, cada paradigma apresenta vantagens estruturais em determinado nível e limitação em outro, configurando *trade-offs* concretos que têm implicações diretas para a escolha de implementações em cenários de microsserviço HTTP.

Sob baixa carga de 100 req/s, as três implementações são equivalentes em latência, e o Clojure Idiomático apresenta a menor pressão sobre o coletor de lixo: 20,67ms de *GC Time* e 1,33 coletas em média, contra 27,5ms e 1,67 coletas do Java. Nesse regime, o custo das abstrações funcionais existe no nível do *bytecode* – 56 ocorrências de *invokestatic* contra 7 do Java –, mas os objetos intermediários gerados pelo *PersistentVector* são coletados de forma eficiente na *Young Generation*, sem pressão sobre o coletor superior à do Java. A escolha entre as três variantes a 100 requisições por segundo é, portanto, indiferente do ponto de vista de latência, com leve vantagem do Clojure Idiomático em GC.

A 500 req/s, o Java apresenta a melhor latência de cauda com P95 de 36,2ms contra 103ms do Clojure Idiomático e 216,2ms do Clojure Interop, evidenciando que a aritmética primitiva direta e a mutação sobre uma única instância de *ArrayList* produzem o perfil de alocação mais favorável nesse regime intermediário. O Clojure Interop, ao reproduzir parcialmente o padrão de mutação do Java mas mantendo infraestrutura de *runtime* Clojure nas demais operações, ocupa posição intermediária em latência mas com *GC Time* superior às demais implementações: 257,6ms contra 167ms do Java e 126,4ms do Clojure Idiomático. O Clojure Idiomático, por sua vez, apresenta o menor *GC Time* a 500 req/s, mas a alocação iterativa de versões intermediárias do *PersistentVector* ainda gera pressão suficiente para comprometer a latência de cauda nesse regime, sem ainda atingir o nível de carga em que essa característica se torna estruturalmente vantajosa. É precisamente aqui que o *trade-off* da imutabilidade se manifesta de forma mais clara: a mesma propriedade que protege a estabilidade a 1000 req/s penaliza a latência a 500 req/s.

A 1000 req/s, as latências médias das execuções estáveis convergiram entre as três implementações, mas o Java demonstrou instabilidade diferencial significativa com CV% de P95 de 31,14% contra 13,05% do Clojure Idiomático e 15,73% do Clojure Interop, evidenciando que a implementação imperativa é estruturalmente mais suscetível ao colapso de GC sob saturação. O Clojure Idiomático, ao manter seus objetos majoritariamente na *Young Generation*, preserva a estabilidade inter-execuções mesmo sob carga extrema, produzindo resultados consistentes entre repetições. O Clojure Interop ocupa posição intermediária: seu padrão de mutação direta sobre *ArrayList* reproduz parcialmente o modo de falha do Java sob pressão extrema de GC, resultando em CV% de P95 superior ao Clojure Idiomático, mas inferior ao Java.

Em síntese, o Java oferece a melhor latência de cauda em regimes de carga moderada mas demonstra instabilidade estrutural crescente sob saturação, com risco diferencial de colapso de GC a 1000 req/s. O Clojure Idiomático apresenta latência equivalente às demais implementações nas execuções estáveis sob alta carga e maior estabilidade inter-execções, ao custo de latência de cauda superior ao Java a 500 req/s. O Clojure Interop constitui a alternativa de menor variância entre regimes extremos, sendo melhor que o Java a 1000 req/s em estabilidade e melhor que o Clojure Idiomático a 500 req/s em latência, sem dominar em nenhum dos dois. Esses *trade-offs* são válidos para o perfil de carga *stateless* investigado, e suas limitações de generalização são discutidas na seção 7.3.

7.3. LIMITAÇÕES DO ESTUDO

Os resultados apresentados neste trabalho foram obtidos em ambiente *single-node*, com todos os containers *Docker* executando sobre a mesma máquina host. Embora o protocolo de execução adotado tenha mitigado as principais fontes de variabilidade ambiental por meio de reinicialização quando necessário e verificação de recursos do sistema antes de cada execução, processos em *background* do sistema operacional e variação térmica sob carga sustentada introduzem ruído residual não completamente eliminável nesse tipo de ambiente. Estudos futuros conduzidos em instâncias de servidor dedicadas, com imagem de sistema operacional mínima e *CPU pinning* dos containers a núcleos físicos específicos, poderiam reduzir essa variabilidade residual e aumentar a reprodutibilidade inter-execções, especialmente nas métricas de *GC Time* e *GC Collections*, que apresentaram os CV% mais elevados do conjunto.

Uma limitação adicional decorre da coexistência do *Gatling* e da aplicação sob teste dentro do mesmo container *Docker*, compartilhando o mesmo orçamento de seis núcleos de CPU configurado para o container. Embora sejam processos JVM independentes, essa coexistência introduz uma variável de contenção de recursos não isolada experimentalmente: não é possível, com a instrumentação atual, distinguir com precisão em que medida picos de latência refletem exclusivamente o comportamento da implementação sob teste, versus uma eventual disputa momentânea por CPU com o próprio gerador de carga. A métrica de CPU Peak disponível reflete apenas o processo da aplicação, não o consumo agregado do container; o fato de essa métrica não ter atingido saturação nas execuções analisadas reduz a plausibilidade de contenção severa e sistemática, mas não a descarta como fonte de ruído residual. Essa condição afetou de forma potencialmente igual as três implementações, dado que a arquitetura de execução foi idêntica em todas, preservando a validade da comparação relativa entre paradigmas – o achado central deste trabalho – ainda que introduza incerteza sobre a magnitude absoluta dos valores de latência reportados. A separação do *Gatling* em um container ou processo dedicado, com orçamento de CPU próprio, constitui direção relevante para trabalhos futuros nesse sentido.

O escopo do estudo está restrito ao processamento de requisições do tipo *stateless*, representado pelo algoritmo de juros compostos: determinístico, independente por requisição e sem compartilhamento de estado entre *threads*. Essa escolha foi metodologicamente justificada pelas propriedades do algoritmo como instrumento de medição, conforme descrito na seção 4.2, mas impede a generalização dos resultados para cenários com estado compartilhado entre requisições, acesso concorrente a recursos externos como banco de dados ou filas de mensagem, ou qualquer *workload* com dependência entre requisições simultâneas. Nesse sentido, é relevante destacar que cenários com concorrência real constituem exatamente o domínio no qual a imutabilidade do Clojure Idiomático potencialmente oferece vantagens estruturais mais pronunciadas, dado que estruturas de dados imutáveis eliminam a necessidade de sincronização entre *threads* e reduzem o risco de *race conditions* (HICKEY, 2020). A investigação do comportamento comparativo dos três paradigmas sob carga com estado compartilhado representa, portanto, uma direção relevante para trabalhos futuros.

Os resultados são específicos à configuração do coletor *G1GC* com *heap* de 4 a 6GB, única configuração avaliada neste estudo. Coletores alternativos disponíveis no *OpenJDK* moderno, como *ZGC* e o *Shenandoah*, possuem arquiteturas fundamentalmente distintas: o *ZGC*, em particular, realiza a maior parte do trabalho de coleta de forma concorrente às *threads* da aplicação, com pausas *Stop-The-World* de duração quase constante e independentes do tamanho do *heap*. Sob um coletor com essas características, o mecanismo causal central identificado neste trabalho – custo diferencial entre *Minor GC* na *Young Generation* e *Mixed GC* ou *Full GC* na *Old Generation* – teria impacto estruturalmente distinto, podendo alterar tanto a magnitude quanto o ordenamento dos resultados observados. A replicação do estudo com configurações alternativas de coletor constitui, portanto, uma extensão natural desta investigação.

Por fim, a hipótese geracional que fundamenta a interpretação causal dos resultados, de que os objetos imutáveis do Clojure Idiomático permanecem majoritariamente na *Young Generation* enquanto os objetos do Java são promovidos prematuramente para *Old Generation* sob alta carga, foi inferida indiretamente a partir das métricas de *GC Time* e *GC Collections* coletadas via *Prometheus*. O estudo não quantificou diretamente a proporção de *Minor GC*, *Mixed GC* e *Full GC* por execução, nem a distribuição de objetos vivos entre gerações do *heap* ao longo do tempo. A hipótese é sustentada pela literatura consolidada sobre o comportamento do *G1GC* (DETLEFS et al., 2004; ZHAO; BLACKBURN, 2020; BRUNO et al., 2017) e é coerente com os padrões empíricos observados, mas sua confirmação direta exigiria instrumentação específica das gerações do *heap* via ferramentas como *JVM Flight Recorder* ou *GC logs* detalhados com a opção `-Xlog:gc`, o que não foi contemplado no escopo deste trabalho.

8. CONSIDERAÇÕES FINAIS

8.1. SÍNTESE DOS ACHADOS

O questionamento da pesquisa formulado na seção 1.2 indaga qual é o custo das abstrações idiomáticas geradas pelo Paradigma Funcional na JVM no processamento de requisições HTTP sob diferentes níveis de carga, e de que forma o comportamento do coletor de lixo medeia tanto a latência observada quanto a estabilidade das implementações sob saturação. Os resultados obtidos permitem responder a esse questionamento de forma direta e estratificada por nível de carga.

A 100 requisições por segundo, as três implementações apresentaram latência P95 equivalente: Java 2,33ms, Clojure Idiomático 2ms e Clojure Interop 2ms, sem diferença operacionalmente relevante entre os paradigmas. O custo das abstrações idiomáticas existe no nível do *bytecode*, com 56 ocorrências de *invokestatic* por requisição contra 7 do Java, mas não se manifesta em degradação de latência observável nesse regime. O comportamento do GC já distingue os paradigmas, dado que o Clojure Idiomático registrou o menor *GC Time* de 20,67ms e 1,33 coletas em média, contra 27,5ms e 1,67 coletas do Java, evidenciando que os objetos intermediários gerados pelo *PersistentVector* são coletados de forma eficiente na *Young Generation* sem pressão adicional sobre o coletor.

A 500 requisições por segundo, o comportamento divergiu, dado que Java registrou o menor P95 de 36,2ms, seguido pelo Clojure Idiomático com 103ms e pelo Clojure Interop com 216,2ms, evidenciando que a aritmética primitiva direta e a mutação sobre uma única instância de *ArrayList* produzem o perfil de alocação mais favorável nesse regime intermediário. Nesse nível de carga, o custo das abstrações idiomáticas se manifesta como penalidade de latência de cauda: a alocação iterativa de versões intermediárias do *PersistentVector* gera pressão crescente sobre o coletor sem ainda atingir o regime em que essa característica se torna estruturalmente vantajosa.

A 1000 requisições por segundo, as latências médias das execuções estáveis convergiram entre as três implementações: Java P95 de 5.502,2ms, Clojure Idiomático 6.781,6ms e Clojure Interop 6.026,4ms, no entanto, a implementação imperativa Java apresentou CV% de P95 de 31,14% contra 13,05% do Clojure Idiomático e 15,73% do Clojure Interop, evidenciando instabilidade diferencial significativa sob saturação. O Java é a única implementação a apresentar colapso nas três rodadas independentes de testes, com execuções inválidas por taxa de erro superior a zero em todos os conjuntos de dados coletados.

O achado central do trabalho é, portanto, a convergência de latência nas execuções estáveis combinada à maior instabilidade estrutural do Java sob saturação. Esse achado é mediado pelo comportamento do coletor de lixo *G1GC*: os objetos imutáveis de vida exígua gerados pelo Clojure Idiomático alinham-se ao pressuposto fundamental da hipótese geracional fraca, segundo o qual objetos recém-alocados têm maior probabilidade de se tornarem lixo, sendo coletados de forma cirúrgica na *Young Generation* com pausas de baixo custo por coleta; o Java, sob alta taxa de injeção sustentada, promove objetos prematuramente para a *Old Generation*, forçando *Mixed GCs* e *Full GCs* com pausas *Stop-The-World* que escalam com o tamanho do *heap* e produzem a dispersão inter-execuções documentada pelo CV%. A cadeia causal central do trabalho – paradigma □ bytecode □ padrão de alocação no *heap* □ comportamento do GC □ latência observada – é, assim, confirmada empiricamente para o perfil de carga *stateless* investigado.

8.2. CONTRIBUIÇÕES

Este trabalho produziu cinco contribuições específicas para a literatura sobre desempenho comparativo entre Java e Clojure na JVM.

A primeira contribuição é a evidência empírica de que o *overhead* de *bytecode* do Clojure Idiomático – 56 ocorrências de *invokestatic* por requisição contra apenas 7 do Java – não se traduz em penalidade de latência nas execuções estáveis sob alta carga. Esse resultado contraria a expectativa intuitiva de que maior densidade de instruções de despacho implica necessariamente maior latência de cauda, e demonstra que o fator determinante sob saturação não é o volume de instruções geradas, mas o padrão de alocação no *heap* e sua interação com o comportamento do coletor geracional.

A segunda contribuição é a documentação do mecanismo de instabilidade diferencial do Java sob saturação via pressão do *Garbage Collector*. O estudo demonstra empiricamente que o CV% de P95 do Java a 1000 req/s – 31,14% contra 13,05% do Clojure Idiomático e 15,73% do Clojure Interop – não reflete variabilidade experimental ou ambiental, mas instabilidade estrutural decorrente da promoção prematura de objetos para a *Old Generation* sob alta taxa de injeção sustentada. Esse mecanismo é distinto de contenção de *locks* ou saturação de *threads*, variáveis confundidoras que não estão presentes no perfil *stateless* das requisições e nem no algoritmo implementado para o cálculo de juros compostos, constituindo, portanto, uma caracterização isolada do efeito da escolha de estrutura de dados mutável sobre a estabilidade do coletor geracional.

A terceira contribuição é a demonstração de que o alinhamento entre o padrão de alocação do Clojure Idiomático e a hipótese geracional fraca do *G1GC* medeia a relação entre paradigma e estabilidade sob carga extrema. A natureza imutável da estrutura de dados *PersistentVector*, ao gerar objetos de vida curta descartados após cada iteração, favorece o pressuposto sobre o qual coletores geracionais modernos foram projetados (UNGAR, 1984; DETLEFS et al., 2004), resultando em coletas cirúrgicas na *Young Generation* com custo por coleta controlado mesmo sob saturação.

A quarta contribuição é a comparação de três variantes paradigmáticas – Java orientado a objetos com *ArrayList* mutável, Clojure Idiomático, funcional, com *PersistentVector* imutável e Clojure Interop com *type hints* e *ArrayList* via interoperabilidade – sob carga HTTP sustentada em três níveis distintos. A lacuna identificada na seção 3 aponta que os trabalhos relacionados investigam pares de linguagens ou paradigmas em *benchmarks* sintéticos, sem contemplar variantes de interoperabilidade dentro de um mesmo ecossistema de *runtime* sob carga de microsserviço real. O Clojure Interop, ao ocupar uma posição intermediária em todos os regimes – melhor que Java em estabilidade a 1000 req/s e melhor que Clojure Idiomático em latência a 500 req/s, sem dominar nenhum cenário – evidencia que a escolha de estrutura de dados dentro de uma mesma linguagem constitui variável de desempenho independente da escolha de linguagem.

A quinta e última contribuição é metodológica: o protocolo de coleta documentado com três rodadas independentes de execução, critérios explícitos de invalidez qualitativa e identificação formal de *outliers* por *z-score* com análise de sensibilidade, constitui uma metodologia replicável para estudos comparativos de desempenho na JVM, endereçando limitações de reprodutibilidade apontadas pela literatura de *benchmarking* (GEORGES et al., 2007; KALIBERA; JONES, 2013).

8.3. TRABALHOS FUTUROS

Os resultados deste trabalho apontam quatro direções prioritárias para investigações futuras.

A primeira é a replicação do experimento em infraestrutura dedicada, com instância de servidor isolada, imagem de sistema operacional mínima e *CPU pinning* dos containers a núcleos físicos específicos. O ambiente *single-node* adotado neste estudo introduz ruído residual não completamente eliminável por processos em *background* e variação térmica sob carga sustentada, especialmente nas métricas de *GC Time* e *GC Collections*, que apresentaram os CV% mais elevados do conjunto. Uma infraestrutura dedicada permitiria quantificar com maior precisão o custo por coleta em cada implementação e fortalecer a confirmação direta da hipótese geracional fraca.

A segunda direção é a extensão do estudo para *workloads* com estado compartilhado e concorrência real, como acesso a banco de dados e filas de mensagens. O perfil *stateless* adotado neste trabalho foi metodologicamente justificado pelo isolamento das variáveis de interesse, mas impede a generalização dos resultados para cenários com dependência entre requisições simultâneas. Esse domínio é precisamente aquele em que a imutabilidade do Clojure Idiomático potencialmente oferece vantagens estruturais mais pronunciadas, dado que estruturas de dados imutáveis eliminam a necessidade de sincronização entre *threads* e reduzem o risco de *race conditions* (HICKEY, 2020).

A terceira direção é a replicação com coletores alternativos disponíveis no *OpenJDK* moderno, em particular o ZGC e o Shenandoah. Esses coletores realizam a maior parte do trabalho de coleta de forma concorrente às *threads* da aplicação, com pausas *Stop-The-World* de duração quase constante e independentes do tamanho do *heap*. Sob essas arquiteturas, o mecanismo causal central identificado neste trabalho — custo diferencial entre *Minor GC* na *Young Generation* e *Mixed GC* ou *Full GC* na *Old Generation* — teria impacto estruturalmente distinto, podendo alterar tanto a magnitude quanto o ordenamento dos resultados observados entre as três implementações.

A quarta direção é a instrumentação direta das gerações do *heap* via *JVM Flight Recorder* ou *GC logs* detalhados com a opção `-Xlog:gc`, permitindo a confirmação direta da hipótese geracional inferida indiretamente neste trabalho a partir das métricas de *GC Time* e *GC Collections*. Essa instrumentação permitiria quantificar a proporção de *Minor GC*, *Mixed GC* e *Full GC* por execução e a distribuição de objetos vivos entre gerações ao longo do tempo, convertendo a hipótese explanatória central de inferência indireta em evidência direta.

9. REFERÊNCIAS BIBLIOGRÁFICAS

ALI, K.; LAI, X.; LUO, Z.; LHOTÁK, O.; DOLBY, J.; TIP, F. **A study of call graph construction for JVM-hosted languages**. *IEEE Transactions on Software Engineering*, New York, v. 47, n. 12, p. 2733-2748, dez. 2021. DOI: 10.1109/TSE.2020.2990823. <https://ieeexplore.ieee.org/document/8944149/>. Último acesso em: 25 abr. 2026.

ALIC, D.; OMANOVIC, S.; GIEDRIMAS, V. **Comparative analysis of functional and object-oriented programming**. In: INTERNATIONAL CONVENTION ON INFORMATION AND COMMUNICATION TECHNOLOGY, ELECTRONICS AND MICROELECTRONICS, 39., 2016, Opatija. *Proceedings...* IEEE, 2016. p. 667-672. DOI: 10.1109/MIPRO.2016.7522224. <https://ieeexplore.ieee.org/document/7522224>. Último acesso em: 18 jul. 2025.

BACKUS, J. W. **Can programming be liberated from the von Neumann style? A functional style and its algebra of programs**. *Communications of the ACM*, New York, v. 21, n. 8, p. 613-641, ago. 1978. <https://dl.acm.org/doi/10.1145/359576.359579>. Último acesso em: 10 abr. 2026.

BRUNO, R.; OLIVEIRA, L.; FERREIRA, P. **NG2C: pretenuring garbage collection with dynamic generations for HotSpot big data applications**. In: ACM SIGPLAN INTERNATIONAL SYMPOSIUM ON MEMORY MANAGEMENT, 2017, Barcelona. *Proceedings...* New York: Association for Computing Machinery, 2017. DOI: 10.1145/3092255.3092272. <https://dl.acm.org/doi/10.1145/3092255.3092272>. Último acesso em: 5 jun. 2026.

DEAN, J.; BARROSO, L. A. **The tail at scale.** Communications of the ACM, New York, v. 56, n. 2, p. 74-80, fev. 2013. DOI: 10.1145/2408776.2408794. <https://dl.acm.org/doi/10.1145/2408776.2408794>. Último acesso em: 1 mai. 2026.

DETLEFS, D.; FLOOD, C.; HELLER, S.; PRINTEZIS, T. **Garbage-first garbage collection.** In: ACM INTERNATIONAL SYMPOSIUM ON MEMORY MANAGEMENT, 2004, Vancouver. Proceedings... New York: Association for Computing Machinery, 2004. DOI: 10.1145/1029873.1029879. <https://dl.acm.org/doi/10.1145/1029873.1029879>. Último acesso em: 3 jun. 2026.

FLOYD, R. W. **The paradigms of programming.** Communications of the ACM, New York, v. 22, n. 8, p. 455-460, ago. 1979. <https://dl.acm.org/doi/10.1145/359138.359140>. Último acesso em: 10 abr. 2026.

FRIEDRICH, S.; WINGERATH, W.; RITTER, N. **Coordinated omission in NoSQL database benchmarking.** In: DATENBANKSYSTEME FÜR BUSINESS, TECHNOLOGIE UND WEB, 17., 2017, Stuttgart. Proceedings... Bonn: Gesellschaft für Informatik, 2017. p. 215-226. <https://dl.gi.de/handle/20.500.12116/918>. Último acesso em: 8 jun. 2026.

GEORGES, A.; BUYTAERT, D.; EECKHOUT, L. **Statistically rigorous Java performance evaluation.** In: ANNUAL ACM SIGPLAN CONFERENCE ON OBJECT-ORIENTED PROGRAMMING SYSTEMS, LANGUAGES AND APPLICATIONS, 22., 2007, Montreal. Proceedings... New York: Association for Computing Machinery, 2007. p. 57-76. DOI: 10.1145/1297027.1297033. <https://dl.acm.org/doi/10.1145/1297027.1297033>. Último acesso em: 7 jun. 2026.

GRUBBS, F. E. **Procedures for detecting outlying observations in samples.** Technometrics, Alexandria, v. 11, n. 1, p. 1-21, fev. 1969. DOI: 10.1080/00401706.1969.10490657. <https://www.tandfonline.com/doi/abs/10.1080/00401706.1969.10490657>. Último acesso em: 7 jun. 2026.

HARRISON, R.; SAMARAWEERA, L. G.; DOBIE, M. R.; LEWIS, P. H. **Comparing programming paradigms: an evaluation of functional and object-oriented programs.** Software Engineering Journal, v. 11, n. 4, p. 247-254, 1996. DOI: 10.1049/sej.1996.0030. <https://digital-library.theiet.org/doi/abs/10.1049/sej.1996.0030>. Último acesso em: 6 jun. 2025.

HICKEY, R. **A history of Clojure.** Proceedings of the ACM on Programming Languages, New York, v. 4, n. OOPSLA, p. 1-46, nov. 2020. DOI: 10.1145/3386326. <https://dl.acm.org/doi/10.1145/3386326>. Último acesso em: 20 abr. 2026.

HOEFLER, T.; BELL, R. **Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results.** In: INTERNATIONAL CONFERENCE FOR HIGH PERFORMANCE COMPUTING, NETWORKING, STORAGE AND ANALYSIS, 2015, Austin. Proceedings... New York: Association for Computing Machinery, 2015. p. 1-12. DOI: 10.1145/2807591.2807644. <https://dl.acm.org/doi/10.1145/2807591.2807644>. Último acesso em: 10 jun. 2026.

HUDAK, P. **Conception, evolution, and application of functional programming languages.** ACM Computing Surveys, New York, v. 21, n. 3, p. 359-411, set. 1989. <https://dl.acm.org/doi/10.1145/72551.72554>. Último acesso em: 13 ago. 2025.

JAIN, R. **The art of computer systems performance analysis: techniques for experimental design, measurement, simulation, and modeling.** New York: Wiley-Interscience, 1991. ISBN: 0-471-50336-3. <https://www.jstor.org/stable/25061650>. Último acesso em: 20 abr. 2026.

KALIBERA, T.; JONES, R. **Rigorous benchmarking in reasonable time.** In: ACM SIGPLAN INTERNATIONAL SYMPOSIUM ON MEMORY MANAGEMENT, 2013, Seattle. Proceedings... New York: Association for Computing Machinery, 2013. p. 63-74. DOI: 10.1145/2464157.2464160. <https://dl.acm.org/doi/10.1145/2464157.2464160>. Último acesso em: 7 jun. 2026.

LACERDA, N. P. java-clojure-performance-analyzer: repositório de código-fonte. 2026. Disponível em: <https://github.com/nathan00pdl/java-clojure-performance-analyzer>. Último acesso em: 25 jul. 2026.

LI, W. H.; WHITE, D. R.; SINGER, J. **JVM-hosted languages: they talk the talk, but do they walk the walk?** In: INTERNATIONAL CONFERENCE ON PRINCIPLES AND PRACTICES OF PROGRAMMING ON THE JAVA PLATFORM: VIRTUAL MACHINES, LANGUAGES, AND TOOLS, 2013, Stuttgart. Proceedings... New York: ACM, 2013. p. 101-112. DOI: 10.1145/2500828.2500838. <https://dl.acm.org/doi/10.1145/2500828.2500838>. Último acesso em: 25 abr. 2026.

LIEBERMAN, H.; HEWITT, C. **A real-time garbage collector based on the lifetimes of objects.** Communications of the ACM, New York, v. 26, n. 6, p. 419-429, jun. 1983. DOI: 10.1145/358141.358147. <https://dl.acm.org/doi/10.1145/358141.358147>. Último acesso em: 3 jun. 2026.

MØLLER, A.; VEILEBORG, O. H. **Eliminating abstraction overhead of Java stream pipelines using ahead-of-time program optimization.** Proceedings of the ACM on Programming Languages, New York, v. 4, n. OOPSLA, p. 1-29, nov. 2020. <https://dl.acm.org/doi/10.1145/3428236>. Último acesso em: 10 abr. 2026.

NAKAMURA, H.; SONOYAMA, A.; KAMIYAMA, T.; OGUCHI, M.; YAMAGUCHI, S. **Performance study of Kotlin and Java programs with bytecode analysis.** Journal of Information Processing, Tokyo, v. 32, p. 380-395, abr. 2024. DOI: 10.2197/ipsjip.32.380. https://www.jstage.jst.go.jp/article/ipsjip/32/0/32_380/article/-char/en. Último acesso em: 20 abr. 2026.

PANKRATIUS, V.; SCHMIDT, F.; GARRETÓN, G. **Combining functional and imperative programming for multicore software: an empirical study evaluating Scala and Java.** In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, 34., 2012, Zurique. Proceedings... IEEE/ACM, 2012. p. 123-133. DOI: 10.1109/ICSE.2012.6227200. <https://ieeexplore.ieee.org/abstract/document/6227200/>. Último acesso em: 14 nov. 2025.

QIU, T.; BLACKBURN, S. M. **Iso: request-private garbage collection.** Proceedings of the ACM on Programming Languages, New York, v. 9, n. PLDI, p. 874-896, jun. 2025. DOI: 10.1145/3729285. <https://dl.acm.org/doi/10.1145/3729285>. Último acesso em: 22 abr. 2026.

ROSE, J. R. **Bytecodes meet combinators: invokedynamic on the JVM.** In: ACM SIGPLAN SYMPOSIUM ON OBJECT-ORIENTED PROGRAMMING SYSTEMS, LANGUAGES AND APPLICATIONS, 2009, Orlando. Proceedings... New York: ACM, 2009. p. 1-11. DOI: 10.1145/1640089.1640109. <https://dl.acm.org/doi/10.1145/1711506.1711508>. Último acesso em: 20 abr. 2026.

SARIMBEKOV, A.; STADLER, L.; BULEJ, L.; SEWE, A.; PODZIMEK, A.; ZHENG, Y.; BINDER, W. **Workload characterization of JVM languages.** Software: Practice and Experience, Hoboken, v. 46, n. 9, p. 1053-1089, 2016. DOI: 10.1002/spe.2337. <https://onlinelibrary.wiley.com/doi/full/10.1002/spe.2337>. Último acesso em: 10 abr. 2026.

SOUSA, B. M. D. de; FERREIRA, R. C.; GOLDMAN, A. **Functional vs. object-oriented: comparing how programming paradigms affect the architectural characteristics of systems.** In: BRAZILIAN SYMPOSIUM ON SOFTWARE QUALITY, 24., 2025, São José dos Campos. Proceedings... New York: Association for Computing Machinery, 2025. p. 584-594. DOI: 10.5753/sbqs.2025.15170. <https://sol.sbc.org.br/index.php/sbqs/article/download/39051/38823/>. Último acesso em: 4 ago. 2025.

TENNAGE, P.; PERERA, S.; JAYASINGHE, M.; JAYASENA, S. **An analysis of holistic tail latency behaviors of Java microservices.** In: IEEE INTERNATIONAL CONFERENCE ON HIGH PERFORMANCE COMPUTING AND COMMUNICATIONS, 21., 2019, Zhangjiajie. Proceedings... IEEE, 2019. p. 697-705. DOI: 10.1109/hpcc/smartcity/dss.2019.00104. <https://ieeexplore.ieee.org/document/8855342>. Último acesso em: 1 mai. 2026.

UNGAR, D. **Generation scavenging: a non-disruptive high performance storage reclamation algorithm.** ACM SIGPLAN Notices, New York, v. 19, n. 5, p. 157-167, maio 1984. DOI: 10.1145/390011.808261. <https://dl.acm.org/doi/10.1145/800020.808261>. Último acesso em: 3 jun. 2026.

WEGNER, P. **Concepts and paradigms of object-oriented programming.** ACM OOPS Messenger, New York, v. 1, n. 1, p. 7-87, ago. 1990. <https://dl.acm.org/doi/10.1145/382192.383004>. Último acesso em: 10 abr. 2026.

WYCIŚLIK, Ł.; LATUSIK, Ł.; KAMIŃSKA, A. M. **A comparative assessment of JVM frameworks to develop microservices.** Applied Sciences, Basel, v. 13, n. 3, p. 1343, jan. 2023. DOI: 10.3390/app13031343. <https://www.mdpi.com/2076-3417/13/3/1343>. Último acesso em: 1 mai. 2026.

ZHAO, W.; BLACKBURN, S. M. **Deconstructing the garbage-first collector.** In: ACM SIGPLAN/SIGOPS INTERNATIONAL CONFERENCE ON VIRTUAL EXECUTION ENVIRONMENTS, 16., 2020, Lausanne. Proceedings... New York: Association for Computing Machinery, 2020. DOI: 10.1145/3381052.3381320. <https://dl.acm.org/doi/10.1145/3381052.3381320>. Último acesso em: 3 jun. 2026.

APÊNDICE A - Dados estatísticos sem outliers por implementação e nível de carga

Carga (req/s)	Métrica	Java Média	Clojure Idiomático Média	Clojure Interop Média	Java CV%	Clojure Idiomático CV%	Clojure Interop CV%
100	P95 (ms)	2,33	2,00	2,00	22,13	0,00	0,00
	P99 (ms)	3,83	4,00	3,80	10,65	0,00	11,77
	GC Time (ms)	27,50	20,67	22,60	38,88	60,25	68,16
	GC Collections	1,67	1,33	1,40	30,98	61,24	63,89
500	P95 (ms)	36,20	103,00	216,20	117,21	124,63	129,69
	P99 (ms)	2.145,60	2.049,00	2.097,00	39,10	40,33	26,80
	GC Time (ms)	167,00	126,40	257,60	64,11	17,18	64,51
	GC Collections	5,60	6,40	5,80	15,97	8,56	7,71
1000	P95 (ms)	5.502,20	6.781,57	6.026,43	31,14	13,05	15,73
	P99 (ms)	10.756,00	11.846,71	10.818,57	12,74	13,42	9,43
	GC Time (ms)	1.024,00	1.295,29	1.132,43	16,23	23,66	12,16
	GC Collections	14,00	17,29	13,86	0,00	9,28	6,49

APÊNDICE B - Bytecode completo: implementação Java Imperativa

Para fins de legibilidade, o prefixo `com.example.java_clojure_performance_analyzer` foi abreviado para `[pkg]` nas referências de classe abaixo.

```
public [pkg].services.JavaCompoundInterestService calculateCompoundInterest(  
    [pkg].models.request.CompoundInterestRequest);
```

Code:

```
    0: aload_1  
  
    1: invokevirtual #18      // Method  
[pkg].request.CompoundInterestRequest.getInitialAmount:()Ljava/lang/Double;  
  
    4: invokevirtual #24      // Method java/lang/Double.doubleValue:()D  
  
    7: dstore_2  
  
    8: aload_1  
  
    9: invokevirtual #30      // Method  
[pkg].request.CompoundInterestRequest.getAnnualInterestRate:()Ljava/lang/Double;  
  
   12: invokevirtual #24      // Method java/lang/Double.doubleValue:()D  
  
   15: dstore          4  
  
   17: aload_1  
  
   18: invokevirtual #33      // Method  
[pkg].request.CompoundInterestRequest.getYears:()Ljava/lang/Integer;  
  
   21: invokevirtual #37      // Method java/lang/Integer.intValue:()I  
  
   24: istore          6  
  
   26: dconst_1  
  
   27: dload          4  
  
   29: ldc2_w         #43      // double 100.0d  
  
   32: ddiv  
  
   33: dadd  
  
   34: dstore          7  
  
   36: new            #45      // class java/util/ArrayList  
  
   39: dup
```

```

40: iload          6
42: invokespecial  #47      // Method java/util/ArrayList."<init>":(I)V
45: astore          9
47: dload_2
48: dstore          10
50: iconst_1
51: istore          12
53: goto           116
56: dload           10
58: dstore          13
60: dload           10
62: dload           7
64: dmul
65: dstore          15
67: dload           15
69: dload           13
71: dsub
72: dstore          17
74: aload           9
76: new             #50      // class [pkg].response.YearlyInvestmentSummary
79: dup
80: iload           12
82: invokestatic  #52      // Method
java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
85: dload           13
87: invokestatic  #56      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;
90: dload           15

```

```

    92: invokestatic #56    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

    95: dload          17

    97: invokestatic #56    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

   100: invokespecial #59    // Method
[pkg].response.YearlyInvestmentSummary."<init>":(Ljava/lang/Integer;Ljava/lang/D
ouble;Ljava/lang/Double;Ljava/lang/Double;)V

   103: invokeinterface #62   // InterfaceMethod
java/util/List.add:(Ljava/lang/Object;)Z

   108: pop

   109: dload          15

   111: dstore         10

   113: iinc           12, 1

   116: iload          12

   118: iload          6

   120: if_icmple      56

   123: dload          10

   125: dload_2

   126: dsub

   127: dstore         12

   129: new            #68    // class [pkg].response.InvestmentSummary

   132: dup

   133: dload_2

   134: invokestatic #56    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

   137: dload          10

   139: invokestatic #56    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

   142: dload          12

   144: invokestatic #56    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

```

```
147: invokespecial #70      // Method
[pkg].response.InvestmentSummary."<init>":(Ljava/lang/Double;Ljava/lang/Double;L
java/lang/Double;)V
```

```
150: astore          14
```

```
152: new              #73      // class [pkg].response.CompoundInterestResponse
```

```
155: dup
```

```
156: aload           14
```

```
158: aload           9
```

```
160: invokespecial #75      // Method
[pkg].response.CompoundInterestResponse."<init>":([pkg].response.InvestmentSumma
ry;Ljava/util/List;)V
```

```
163: areturn
```

LineNumberTable:

```
line 17: 0
```

```
line 18: 8
```

```
line 19: 17
```

```
line 21: 26
```

```
line 23: 36
```

```
line 25: 47
```

```
line 27: 50
```

```
line 28: 56
```

```
line 29: 60
```

```
line 30: 67
```

```
line 32: 74
```

```
line 34: 109
```

```
line 27: 113
```

```
line 37: 123
```

```
line 38: 129
```

```
line 40: 152
```

LocalVariableTable:

Start	Length	Slot	Name	Signature
0	164	0	this	
L[pkg].services.JavaCompoundInterestService;				
0	164	1	request	
L[pkg].models.request.CompoundInterestRequest;				
8	156	2	initialAmount	D
17	147	4	rate	D
26	138	6	years	I
36	128	7	compoundFactor	D
47	117	9	yearlyDetailsList	Ljava/util/List;
50	114	10	currentAmount	D
53	70	12	year	I
60	53	13	startBalance	D
67	46	15	endBalance	D
74	39	17	interestEarned	D
129	35	12	totalInterestEarned	D
152	12	14	summary	
L[pkg].response.InvestmentSummary;				

APÊNDICE C - Bytecode completo: implementação Clojure Idiomático

Para fins de legibilidade, o prefixo `com.example.java_clojure_performance_analyzer` foi abreviado para `[pkg]` e o prefixo `clojure.lang` foi abreviado para `[clj]` nas referências de classe abaixo.

```
public static java.lang.Object invokeStatic(java.lang.Object);
```

Code:

```
0: aload_0

1: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

4: invokevirtual #17    // Method
[pkg].models.request.CompoundInterestRequest.getInitialAmount:()Ljava/lang/Double;

7: astore_1

8: aload_0

9: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

12: invokevirtual #20    // Method
[pkg].models.request.CompoundInterestRequest.getAnnualInterestRate:()Ljava/lang/Double;

15: astore_2

16: aload_0

17: aconst_null

18: astore_0

19: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

22: invokevirtual #24    // Method
[pkg].models.request.CompoundInterestRequest.getYears:()Ljava/lang/Integer;

25: astore_3

26: dconst_1

27: aload_2

28: aconst_null
```

```

29: astore_2

30: ldc2_w      #25      // double 100.0d

33: invokestatic #32      // Method
[clj].Numbers.divide:(Ljava/lang/Object;D)D

36: dadd

37: dstore       4

39: new          #34      // class [pkg].service...fn__275

42: dup

43: aload_1

44: aload_3

45: aconst_null

46: astore_3

47: dload        4

49: invokespecial #37      // Method
[pkg].service...fn__275."<init>":(Ljava/lang/Object;Ljava/lang/Object;D)V

52: checkcast    #39      // class [clj].IFn

55: invokeinterface #43    // InterfaceMethod
[clj].IFn.invoke:()Ljava/lang/Object;

60: astore       6

62: getstatic    #47      // Field const__4:L[clj].Var;

65: invokevirtual #52      // Method
[clj].Var.getRawRoot:()Ljava/lang/Object;

68: checkcast    #39      // class [clj].IFn

71: aload        6

73: invokeinterface #54    // InterfaceMethod
[clj].IFn.invoke:(Ljava/lang/Object;)Ljava/lang/Object;

78: dup

79: ifnull       92

82: getstatic    #60      // Field
java/lang/Boolean.FALSE:Ljava/lang/Boolean;

85: if_acmpeq    93

```

```
88: aload_1

89: goto          115

92: pop

93: getstatic      #63      // Field const__5:L[clj].Var;

96: invokevirtual #52      // Method
[clj].Var.getRawRoot:()Ljava/lang/Object;

99: checkcast      #39      // class [clj].IFn

102: aload          6

104: invokeinterface #54     // InterfaceMethod
[clj].IFn.invoke:(Ljava/lang/Object;)Ljava/lang/Object;

109: checkcast      #65      // class [pkg].response.YearlyInvestmentSummary

112: invokevirtual #68      // Method
[pkg].response.YearlyInvestmentSummary.getEndBalance:()Ljava/lang/Double;

115: astore         7

117: aload          7

119: aload_1

120: invokestatic   #72      // Method
[clj].Numbers.minus:(Ljava/lang/Object;Ljava/lang/Object;)Ljava/lang/Number;

123: astore         8

125: new            #74      // class [pkg].response.InvestmentSummary

128: dup

129: aload_1

130: aconst_null

131: astore_1

132: checkcast      #76      // class java/lang/Double

135: aload          7

137: aconst_null

138: astore         7

140: checkcast      #76      // class java/lang/Double

143: aload          8
```

```

145: aconst_null

146: astore          8

148: checkcast       #76    // class java/lang/Double

151: invokespecial   #79    // Method
[pgk].response.InvestmentSummary."<init>":(Ljava/lang/Double;Ljava/lang/Double;L
java/lang/Double;)V

154: astore          9

156: new             #81    // class [pgk].response.CompoundInterestResponse

159: dup

160: aload           9

162: aconst_null

163: astore          9

165: checkcast       #74    // class [pgk].response.InvestmentSummary

168: aload           6

170: aconst_null

171: astore          6

173: checkcast       #83    // class java/util/List

176: invokespecial   #86    // Method
[pgk].response.CompoundInterestResponse."<init>":([pgk].response.InvestmentSumma
ry;Ljava/util/List;)V

179: areturn

```

LineNumberTable:

```

line 7: 0

line 9: 4

line 10: 12

line 11: 22

line 13: 33

line 13: 36

line 15: 52

line 15: 55

```

line 43: 62

line 43: 68

line 43: 73

line 45: 99

line 45: 104

line 45: 112

line 48: 120

LocalVariableTable:

Start	Length	Slot	Name	Signature
8	171	1	initial_amount	Ljava/lang/Object;
16	163	2	rate	Ljava/lang/Object;
26	153	3	years	Ljava/lang/Object;
39	140	4	compound_factor	D
62	117	6	yearly_details_list	Ljava/lang/Object;
117	62	7	final_amount	Ljava/lang/Object;
125	54	8	total_interest_earned	Ljava/lang/Object;
156	23	9	summary	Ljava/lang/Object;
0	179	0	request	Ljava/lang/Object;

```
public java.lang.Object invoke();
```

Code:

0: lconst_1

1: lstore_1

2: aload_0

3: getfield #17 // Field initial_amount:Ljava/lang/Object;

6: aload_0

7: aconst_null

```

    8: putfield      #17    // Field initial_amount:Ljava/lang/Object;
11: astore_3

    12: getstatic    #29    // Field
[clj].PersistentVector.EMPTY:L[clj].PersistentVector;

    15: astore      4

    17: lload_1

    18: aload_0

    19: getfield      #19    // Field years:Ljava/lang/Object;

    22: invokestatic  #35    // Method [clj].Numbers.gt:(Ljava/lang/Object;)Z

    25: ifeq          37

    28: aload         4

    30: aconst_null

    31: astore        4

    33: goto          149

    36: athrow

    37: aload_3

    38: astore        5

    40: aload_3

    41: aconst_null

    42: astore_3

    43: aload_0

    44: getfield      #21    // Field compound_factor:D

    47: invokestatic  #39    // Method
[clj].Numbers.multiply:(Ljava/lang/Object;)D

    50: dstore        6

    52: dload         6

    54: aload         5

    56: invokestatic  #43    // Method
[clj].Numbers.minus:(DLjava/lang/Object;)D

```

```

59: dstore      8

61: lload_1

62: invokestatic #47    // Method [clj].Numbers.inc:(J)J

65: dload      6

67: invokestatic #53    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

70: getstatic   #57    // Field const__5:L[clj].Var;

73: invokevirtual #62    // Method
[clj].Var.getRawRoot:()Ljava/lang/Object;

76: checkcast   #64    // class [clj].IFn

79: aload      4

81: aconst_null

82: astore      4

84: new         #66    // class [pkg].response.YearlyInvestmentSummary

87: dup

88: lload_1

89: invokestatic #72    // Method [clj].RT.intCast:(J)I

92: invokestatic #77    // Method
java/lang/Integer.valueOf:(I)Ljava/lang/Integer;

95: checkcast   #74    // class java/lang/Integer

98: aload      5

100: aconst_null

101: astore      5

103: invokestatic #81    // Method
[clj].RT.doubleCast:(Ljava/lang/Object;)D

106: invokestatic #53    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

109: checkcast   #49    // class java/lang/Double

112: dload      6

114: invokestatic #84    // Method [clj].RT.doubleCast:(D)D

```

```

    117: invokestatic #53      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

    120: checkcast      #49      // class java/lang/Double

    123: dload          8

    125: invokestatic #84      // Method [clj].RT.doubleCast:(D)D

    128: invokestatic #53      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

    131: checkcast      #49      // class java/lang/Double

    134: invokespecial #87      // Method
[pkg].response.YearlyInvestmentSummary."<init>":(Ljava/lang/Integer;Ljava/lang/D
ouble;Ljava/lang/Double;Ljava/lang/Double;)V

    137: invokeinterface #90     // InterfaceMethod
[clj].IFn.invoke:(Ljava/lang/Object;Ljava/lang/Object;)Ljava/lang/Object;

    142: astore          4

    144: astore_3

    145: lstore_1

    146: goto           17

    149: areturn

```

LineNumberTable:

```

    line 15: 0

    line 20: 17

    line 20: 22

    line 24: 47

    line 25: 56

    line 28: 62

    line 30: 76

    line 32: 89

    line 33: 103

    line 34: 114

    line 35: 125

```


line 30: 137

LocalVariableTable:

Start	Length	Slot	Name	Signature
40	109	5	start_balance	Ljava/lang/Object;
52	97	6	end_balance	D
61	88	8	interest_earned	D
2	147	1	year	J
12	137	3	current_amount	Ljava/lang/Object;
17	132	4	yearly_details_list	Ljava/lang/Object;
0	149	0	this	Ljava/lang/Object;

Para fins de legibilidade, o prefixo com.example.java_clojure_performance_analyzer foi abreviado para [pkg] e o prefixo clojure.lang foi abreviado para [clj] nas referências de classe abaixo.

```
public static java.lang.Object invokeStatic(java.lang.Object);
```

Code:

```
0: aload_0

1: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

4: invokevirtual #17    // Method
[pkg].models.request.CompoundInterestRequest.getInitialAmount:()Ljava/lang/Double;

7: invokestatic #23    // Method
[clj].RT.doubleCast:(Ljava/lang/Object;)D

10: dstore_1

11: aload_0

12: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

15: invokevirtual #26    // Method
[pkg].models.request.CompoundInterestRequest.getAnnualInterestRate:()Ljava/lang/Double;

18: invokestatic #23    // Method
[clj].RT.doubleCast:(Ljava/lang/Object;)D

21: dstore_3

22: aload_0

23: aconst_null

24: astore_0

25: checkcast    #13    // class
[pkg].models.request.CompoundInterestRequest

28: invokevirtual #30    // Method
[pkg].models.request.CompoundInterestRequest.getYears:()Ljava/lang/Integer;

31: invokestatic #34    // Method [clj].RT.intCast:(Ljava/lang/Object;)I

34: istore      5

36: dconst_1
```

```

37: dload_3

38: ldc2_w      #35      // double 100.0d

41: ddiv

42: dadd

43: dstore      6

45: new          #38      // class java/util/ArrayList

48: dup

49: iload        5

51: invokespecial #41      // Method java/util/ArrayList."<init>":(I)V

54: astore      8

56: new          #43      // class [pkg].service...fn__285

59: dup

60: aload        8

62: iload        5

64: dload        6

66: dload_1

67: invokespecial #46      // Method
[pkg].service...fn__285."<init>":(Ljava/lang/Object;IDD)V

70: checkcast    #48      // class [clj].IFn

73: invokeinterface #52    // InterfaceMethod
[clj].IFn.invoke:()Ljava/lang/Object;

78: astore      9

80: aload        9

82: dload_1

83: invokestatic #58      // Method
[clj].Numbers.minus:(Ljava/lang/Object;D)D

86: dstore      10

88: new          #60      // class [pkg].response.InvestmentSummary

91: dup

```

```

92: dload_1

93: invokestatic #66    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

96: checkcast      #62    // class java/lang/Double

99: aload          9

101: aconst_null

102: astore          9

104: checkcast      #62    // class java/lang/Double

107: dload          10

109: invokestatic #66    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;

112: checkcast      #62    // class java/lang/Double

115: invokespecial #69    // Method
[pkg].response.InvestmentSummary."<init>":(Ljava/lang/Double;Ljava/lang/Double;L
java/lang/Double;)V

118: astore          12

120: new             #71    // class [pkg].response.CompoundInterestResponse

123: dup

124: aload          12

126: aconst_null

127: astore          12

129: checkcast      #60    // class [pkg].response.InvestmentSummary

132: aload          8

134: aconst_null

135: astore          8

137: checkcast      #73    // class java/util/List

140: invokespecial #76    // Method
[pkg].response.CompoundInterestResponse."<init>":([pkg].response.InvestmentSumma
ry;Ljava/util/List;)V

143: areturn

```

LineNumberTable:

line 8: 0
line 10: 4
line 10: 7
line 11: 15
line 11: 18
line 12: 28
line 12: 31
line 14: 41
line 14: 42
line 19: 70
line 19: 73
line 44: 83

LocalVariableTable:

Start	Length	Slot	Name	Signature
11	132	1	initial_amount	D
22	121	3	rate	D
36	107	5	years	I
45	98	6	compound_factor	D
56	87	8	yearly_details_list	Ljava/lang/Object;
80	63	9	final_amount	Ljava/lang/Object;
88	55	10	total_interest_earned	D
120	23	12	summary	Ljava/lang/Object;
0	143	0	request	Ljava/lang/Object;

```
public java.lang.Object invoke();
```

Code:

```
0: lconst_1
```

```

1: lstore_1
2: aload_0
3: getfield      #25    // Field initial_amount:D
6: dstore_3
7: lload_1
8: aload_0
9: getfield      #21    // Field years:I
12: i2l
13: lcmp
14: ifle         25
17: dload_3
18: invokestatic #33    // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;
21: goto         105
24: athrow
25: dload_3
26: dstore        5
28: dload_3
29: aload_0
30: getfield      #23    // Field compound_factor:D
33: dmul
34: dstore        7
36: dload         7
38: dload         5
40: dsub
41: dstore        9
43: aload_0
44: getfield      #19    // Field yearly_details_list:Ljava/lang/Object;

```

```

47: checkcast      #35      // class java/util/ArrayList
50: new             #37      // class [pkg].response.YearlyInvestmentSummary
53: dup
54: lload_1
55: invokestatic    #43      // Method [clj].RT.intCast:(J)I
58: invokestatic    #48      // Method
java/lang/Integer.valueOf:(I)Ljava/lang/Integer;
61: checkcast      #45      // class java/lang/Integer
64: dload           5
66: invokestatic    #33      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;
69: checkcast      #29      // class java/lang/Double
72: dload           7
74: invokestatic    #33      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;
77: checkcast      #29      // class java/lang/Double
80: dload           9
82: invokestatic    #33      // Method
java/lang/Double.valueOf:(D)Ljava/lang/Double;
85: checkcast      #29      // class java/lang/Double
88: invokespecial   #51      // Method
[pkg].response.YearlyInvestmentSummary."<init>":(Ljava/lang/Integer;Ljava/lang/D
ouble;Ljava/lang/Double;Ljava/lang/Double;)V
91: invokevirtual   #55      // Method
java/util/ArrayList.add:(Ljava/lang/Object;)Z
94: pop
95: lload_1
96: lconst_1
97: ladd
98: dload           7
100: dstore_3

```

101: lstore_1

102: goto 7

105: areturn

LineNumberTable:

line 19: 0

line 22: 7

line 22: 7

line 26: 33

line 27: 40

line 32: 55

line 30: 91

line 38: 96

LocalVariableTable:

Start	Length	Slot	Name	Signature
28	77	5	start_balance	D
36	69	7	end_balance	D
43	62	9	interest_earned	D
2	103	1	year	J
7	98	3	current_amount	D
0	105	0	this	Ljava/lang/Object;